Design and Validation of a Reproducible C++/Python Stochastic Risk Engine

QuantRisk++ — Technical Report

Phase 10 deliverable, revision `02fea3a` (`git log`), library version 0.1.0

Platform of every recorded measurement: AppleClang 21.0.0.21000334, arm64, macOS (Darwin 27.0.0),
CPython 3.12.14, CMake 4.4.3, QuantLib 1.43, SciPy 1.18.1, PyPortfolioOpt 1.6.0, cvxpy 1.9.3, scikit-learn 1.9.1

Abstract

This report describes the design, implementation and validation of QuantRisk++, a C++20 numerical core with a Python research interface covering option pricing, Monte Carlo simulation, market-risk measurement, portfolio optimisation and scenario stress testing. The organising commitment of the project is that no claim about correctness is made without an executable artifact behind it. Three validation levels are defined before any code exists and are then executed in every phase: analytical identities and degenerate limits, comparison against independently written libraries run live, and statistical studies of convergence, size, power and coverage on synthetic data whose truth is known in closed form. The library implements its own numerics throughout; QuantLib, PyPortfolioOpt, cvxpy, SciPy and scikit-learn are used only as correctness oracles and are imported by no library code, so agreement with an oracle is evidence about the implementation rather than a restatement of the oracle's own answer. Sections 9–10 report what was measured: agreement with a live analytic engine to 1.49×10^{-13} absolute on 2,464 Black–Scholes price comparisons, fitted Monte Carlo convergence slopes of -0.61 to -0.51 against a theoretical -0.5, a Kupiec test with 5.05% empirical size at a 5% nominal level, and portfolio solvers agreeing with two independent reference solver stacks to 2.4×10^{-12} in weights on the quadratic problems. Sections 11–12 report what was not established: 96 recorded limitations, the fact that every figure in this report is computed from synthetic data-generating processes authored by the same person who wrote the code, that no number here describes realised markets, and that reproducibility is per platform rather than bit-identical across mathematical libraries. Several defects found by the validation process are described in full, because their discovery is the strongest available evidence that the process works.

A note on the numbers in this report. Every quantitative statement below is copied from a file in this repository, and the file is named in `brackets` immediately beside it. Numbers that could not be traced to a committed artifact were removed from the text rather than estimated. Percentages and error bounds are reproduced at the precision at which the artifact records them, or one digit coarser. The artifacts themselves are regenerated by the commands listed in Section 12.

Contents

1 Introduction **4**

1.1 What this report does and does not claim 5

2 Mathematical Background **6**

2.1 Measure, units and the loss convention 6

2.2 Diffusions and closed forms 6

2.3 Risk measures 7

2.4 Portfolio and optimisation 7

2.5 Path-dependent payoffs 8

3 Software Architecture **8**

3.1 Layers and permitted direction of dependency 8

3.2 The Python face, and how it is prevented from computing anything 9

3.3 Interface freeze 9

3.4 Build, packaging and dependency policy 9

3.5 Foundational numerics 10

3.6 Optional data layer 10

4 Pricing Models **10**

4.1 Black–Scholes–Merton 10

4.2 Greeks by finite differences 11

4.3 The lattice 11

4.4 Asian and barrier options, and where discretisation actually costs 13

4.5 Heston simulation 14

5 Monte Carlo Methods **14**

5.1 Convergence and standard-error calibration 14

5.2 Variance reduction, measured out of sample 15

6 Market Risk Estimation **16**

6.1 Point estimators against known population values 16

6.2 Coverage tests: size, power and what each can see 17

6.3 Uncertainty by resampling 18

6.4 Do the conclusions survive real data? 18

7 Portfolio Optimization **19**

7.1 Agreement with independent solvers 20
7.2 What estimating a covariance costs 21

8 Stress Testing **23**
8.1 Closure, independent numerics, and the measured truncation error 23
8.2 The finding: stress inverts the safety ranking 24
8.3 A remainder bound that predicts a sign change 26
8.4 The same bound when two factors move 27
8.5 Re-striking gamma: the recommendation, measured 29
8.6 The fourth order: a wider crossing radius, and no cheaper amount 30

9 Numerical Validation **32**
9.1 How tolerances are set 32
9.2 Validation matrix 33
9.3 Defects found by the process 33
9.4 What the process did not catch 36

10 Performance Evaluation **37**

11 Limitations **38**
11.1 Distribution of the register 39

12 Reproducibility **39**
12.1 Per-platform, not bit-identical everywhere 41
12.2 What is recorded with a result 42
12.3 The artifact chain, and how to walk it 42
12.4 Known weak points in the reproducibility story 43
12.5 Reproducibility as a documentation property 43

Artifact index **45**

1 Introduction

Quantitative software is easy to write and hard to trust. A pricing routine that returns a plausible number for every input the author tried is indistinguishable, from the outside, from one that is correct; the difference appears only at an input nobody thought to test, or in a tail nobody observed. The premise of this project is that the difference can be made visible at build time, and kept visible, by attaching an executable piece of evidence to every claim the software makes about itself.

QuantRisk++ implements a numerical core in C++20 (`cpp/`), exposes it to Python through thin bindings (`bindings/python_bindings.cpp`), and layers on top of that a Python research interface, an experiment harness, a benchmark harness and a documentation set (`README.md`; `docs/architecture.md`). The functional coverage is

- closed-form European pricing with continuous dividends, analytic and finite-difference Greeks, and a Cox–Ross–Rubinstein lattice (`docs/phase_reports/phase-02-deterministic-pricing.md`);
- Monte Carlo pricing with antithetic and control-variate estimators, arithmetic and geometric Asians and discretely monitored barriers (`docs/phase_reports/phase-03-monte-carlo.md`; `docs/phase_reports/phase-04-path-dependent-heston.md`);
- Heston simulation by full-truncation Euler (`docs/model_cards/heston.md`);
- historical, Gaussian and Monte Carlo VaR and expected shortfall, with bootstrap intervals and coverage tests (`docs/phase_reports/phase-05-market-risk.md`);
- three covariance estimators and five portfolio objectives, each solution carrying a certificate checked against its own inputs (`docs/phase_reports/phase-06-portfolio-optimisation.md`);
- a stress layer mapping factor exposures to profit and loss, attributed per factor and per position (`docs/phase_reports/phase-07-stress-testing.md`);
- an optional offline-capable public-data layer (`docs/phase_reports/phase-08-public-data.md`); and
- five Python facades and a three-command CLI (`docs/phase_reports/phase-09-research-api.md`).

This report describes the repository at **v1.4.0**. Phases 1 to 10 are recorded complete with their phase gates passed, and four later phases carry no PROJECT_SPEC number because they answer `docs/portfolio_audit.md` or this repository's own debt list rather than the specification: Phase 11 is the real-data study of Section 6.4, Phase 12 the single-factor remainder bound of Section 8.3, Phase 13 the two-factor bound of Section 8.4, which converts the refusal Phase 12 recorded into a measured result, and Phase 14 the verification guards of Section 12, which make a figure, a compiled name or a declared path in the prose impossible to state without an owner. Nine release tags exist, `v1.0.0`, `v1.1.0`, `v1.2.0`, `v1.3.0`, `v1.4.0`, `v1.5.0`, `v1.6.0`, `v1.7.0` and `v1.8.0`, each with the report, the evidence manifest and the suite results attached. The suite is 204 CTest entries and 548,368 assertions in 203 Catch2 cases on the C++ side, and 514 pytest tests with the validation oracles

installed, 445 collected without them (`docs/limitations.md`, item 63, explains why one command gives two numbers); 19 benchmark-suite members execute under `--require-all`, and the frozen manifest hashes 95 artifacts with nothing changed, missing or unlisted (`evidence/manifest.json`). The *end-of-Phase-9* baseline, for contrast, was 190 CTest entries and 310 pytest tests with no release tag and no suite at all (`docs/phase_reports/phase-09-research-api.md`). The limitations register contains 96 numbered entries (`docs/limitations.md`).

Two design decisions shape everything that follows, and both are stated in the repository before the corresponding code exists.

The library implements its own numerics; oracles verify, they do not provide. QuantLib [the QuantLib contributors, 2024], PyPortfolioOpt [Martin, 2024], cvxpy [the cvxpy development team, 2024] (with the OSQP and SCS solvers), SciPy [Virtanen et al., 2020] and scikit-learn [Pedregosa et al., 2011] appear in `pyproject.toml` only under `[project.optional-dependencies]`, in the group named `oracles`. A search of the shipped package (`python/quantrisk/`), the C++ sources (`cpp/`) and the bindings at this revision finds no import or link of any of them; the imports live in `benchmarks/`, `experiments/var_backtesting/run.py` and `tests/python/`, where the oracle modules are pulled in through `pytest.importorskip` so that an environment without them skips those tests instead of failing (`pyproject.toml`; `docs/validation_protocol.md` §6). The reason this matters is epistemic. If the library delegated pricing to an oracle and were then tested against that oracle, the test would verify the wiring, not the mathematics. Keeping the oracles outside the dependency graph of the code under test means a disagreement is a real disagreement about the world, and the record contains several such disagreements (Section 9). The rule that follows from it — no oracle output is ever pasted into a test as an expected value — is stated in `docs/validation_protocol.md` and is what makes Section 9 a report of measurements rather than of transcriptions.

Correctness is a per-phase gate, not a final cleanup. `PROJECT_SPEC.md` §2 ranks correctness above features and §4 forbids fabricated benchmarks, performance improvements, test coverage, tolerances and conclusions, forbids deleting failing tests or quietly lowering a tolerance to make a suite pass, and forbids hard-coding an oracle value as an expected result. Each phase report therefore ends with a gate table and a technical-debt list, and each phase writes its failures into `docs/limitations.md` rather than removing them. Section 9 presents the defects found this way, because a validation regime that found nothing across nine phases would be far less credible than one that found the six C++ failures listed in `docs/phase_reports/phase-05-market-risk.md`.

1.1 What this report does and does not claim

The claims are: that the implemented formulas agree with closed forms and with independently written libraries to stated, measured precision; that the statistical behaviour of the stochastic and risk machinery is what theory predicts on synthetic processes whose truth is known; and that every number is regenerable by a recorded command on the recorded platform. The claims are *not*: that any model is calibrated to market data, that any risk measure is accurate for a real portfolio, or that the software outperforms existing libraries.

Section 11 explains why those stronger statements are unavailable, and Section 12 why "reproducible here" is not the same statement as "reproducible everywhere".

2 Mathematical Background

Conventions are fixed in `docs/mathematical_specification.md` and are normative for the code; deviations from them are recorded as limitations rather than silently absorbed.

2.1 Measure, units and the loss convention

Time is measured in years as supplied year fractions, with no day-count inference inside the numerical core (`docs/mathematical_specification.md` §0); rates r and q and volatility σ are annualised, the first two continuously compounded. The risk-neutral measure $\mathbb{Q}$ is used for pricing and the physical measure $\mathbb{P}$ for risk, and the separation is enforced by the API rather than by comment: the risk-neutral generator and `terminal_prices_physical`, whose `mu` denotes price appreciation *excluding* the dividend yield, are different functions (`docs/phase_reports/phase-03-monte-carlo.md`). The loss convention is frozen as $L = -R$, so larger is worse everywhere in the risk and portfolio layers (`docs/mathematical_specification.md` §6). Greek units were not assumed either: the comparison against `benchmarks/quantlib/results/pricing_vs_quantlib.json` measured that the oracle's `vega()` and `rho()` use the same per-unit convention as this build, so no rescaling is applied (`docs/phase_reports/phase-02-deterministic-pricing.md`).

2.2 Diffusions and closed forms

Under $\mathbb{P}$ and $\mathbb{Q}$ respectively, the underlying follows

$$dS_t = \mu S_t\,dt + \sigma S_t\,dW_t^{\mathbb{P}}, \qquad dS_t = (r-q)S_t\,dt + \sigma S_t\,dW_t^{\mathbb{Q}}, \tag{1}$$

with the exact lognormal transition $S_{t+dt} = S_t \exp[(\mu-\sigma^2/2)dt+\sigma\sqrt{dt}\,Z]$, $Z \sim N(0,1)$, used by the simulation layer (`docs/mathematical_specification.md` §1). For $T > 0$, $\sigma > 0$, European prices are [Black and Scholes, 1973]

$$d_1 = \frac{\ln(S/K) + (r - q + \sigma^2/2)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}, \tag{2}$$

$$C = Se^{-qT}N(d_1) - Ke^{-rT}N(d_2), \qquad P = Ke^{-rT}N(-d_2) - Se^{-qT}N(-d_1), \tag{3}$$

with put–call parity $C - P = Se^{-qT} - Ke^{-rT}$ required of every implementation, and the edge cases $T = 0$ (intrinsic) and $\sigma = 0$ (discounted intrinsic of the forward) implemented and tested (`docs/mathematical_specification.md` §2). Greeks are the standard analytic expressions with Theta per calendar year and Vega and Rho per unit of the respective parameter (`docs/mathematical_specification.md` §3).

The lattice is [Cox et al., 1979] with

$$u = e^{\sigma\sqrt{dt}}, \qquad d = 1/u, \qquad p = \frac{e^{(r-q)dt} - d}{u - d}, \qquad dt = T/N; \tag{4}$$

node levels are computed as $Se^{(2j-k)\ln u}$ rather than multiplicatively, so no drift accumulates in the level arithmetic (`docs/phase_reports/phase-02-deterministic-pricing.md`). American exercise takes the maximum of exercise and continuation at each node.

Stochastic volatility is the Heston model [Heston, 1993],

$$dS_t = (r-q)S_t\,dt + \sqrt{v_t}\,S_t\,dW_t^1, \qquad dv_t = \kappa(\theta - v_t)\,dt + \xi\sqrt{v_t}\,dW_t^2, \qquad \operatorname{corr}(dW^1, dW^2) = \rho, \tag{5}$$

with the Feller condition $2\kappa\theta \geq \xi^2 \geq 0$ *reported* on every result and never enforced, because a violating parameter set is an admissible choice and the full-truncation scheme exists precisely because that regime is reachable (`docs/model_cards/heston.md`).

2.3 Risk measures

For confidence level $\alpha \in (0,1)$ applied to the loss L,

$$\mathrm{VaR}_\alpha = \inf\{l : \Pr(L \leq l) \geq \alpha\}, \qquad \mathrm{ES}_\alpha = \mathbb{E}[L \mid L \geq \mathrm{VaR}_\alpha]. \tag{6}$$

VaR is not subadditive and therefore not a coherent risk measure [Artzner et al., 1999]; ES is. That distinction is recorded as a limitation of the layer rather than glossed (`docs/mathematical_specification.md` §6). Empirical VaR uses the linear-interpolated quantile with the NumPy `method="linear"` convention, $h = p(n-1)$ between order statistics, and empirical ES the mean of the $k = \lceil n(1-\alpha) \rceil$ largest losses (`docs/model_cards/var_es.md`). Gaussian variants are $-\hat{\mu} + z_\alpha \hat{\sigma}$ and $-\hat{\mu} + \hat{\sigma}\phi(z_\alpha)/(1-\alpha)$ with $\hat{\sigma}$ the unbiased sample standard deviation.

Coverage is tested with the likelihood-ratio statistics of Kupiec [1995] and Christoffersen [2003], implemented as

$$LR_{po} = -2\ln\frac{(1-p)^{n-x}p^{x}}{(1-\hat{x})^{n-x}\hat{x}^{x}}, \qquad p = 1-\alpha, \quad \hat{x} = x/n, \tag{7}$$

against $\chi^2(1)$, where each count term is evaluated through a helper that applies the $0\log(0) \to 0$ limit rather than arithmetic (`cpp/src/risk/backtest.cpp`). The independence statistic compares a restricted single-transition-probability fit against an unrestricted two-state Markov fit on the four transition counts π_{01}, π_{11}, also on $\chi^2(1)$, and conditional coverage is their sum on $\chi^2(2)$, which is a valid test only because the two components are asymptotically orthogonal (`docs/phase_reports/phase-05-market-risk.md` §2).

2.4 Portfolio and optimisation

The mean–variance problem is $\min_w \frac{1}{2} w^\top \Sigma w$ subject to $Aw = b$, $w \geq 0$, with $A = [\mathbf{1}, \mu]$ when a target return is requested and $A = [\mathbf{1}]$ for the minimum-variance solution; $\Sigma \succeq 0$ gives convexity, while uniqueness of the argument needs $\Sigma \succ 0$ (`docs/mathematical_specification.md` §7). Maximum Sharpe $\max_w (\mu^\top w - r_f)/\sqrt{w^\top \Sigma w}$ is explicitly *not* a quadratic programme, and the specification requires the reformulation used to be documented rather than assumed; it is stated in the returned `note` field (`docs/model_cards/portfolio_covariance_and_optimisation.md`). Conditional value-at-risk is handled as the linear programme of Rockafellar and Uryasev [2000],

$$\min_{w,\alpha,u} \; \alpha + \frac{1}{(1-\beta)M}\sum_{i=1}^{M} u_i \quad \text{s.t.} \quad u_i \geq L_i(w) - \alpha,\; u_i \geq 0,\; \mathbf{1}^\top w = 1,\; w \geq 0, \tag{8}$$

with α split as $a^+ - a$ so the programme stays in standard form, and the tail weight $1-\beta$ interpolated when $(1-\beta)M$ is not an integer, matching the quantile convention used elsewhere (`docs/mathematical_specification.md` §8; `docs/phase_reports/phase-06-portfolio-optimisation.md`). Equal risk contribution is defined by $w_i(\Sigma w)_i = \frac{1}{n} w^\top \Sigma w$ for all i and solved by cyclic coordinate descent on $\frac{1}{2} w^\top \Sigma w - c \sum_i \log w_i$; the reference route in the literature is the convex formulation of Spinu [2013], which is used as an oracle path and not as our solver (`docs/mathematical_specification.md` §9; `docs/model_cards/portfolio_covariance_and_optimisation.md`).

Covariance is estimated three ways: unbiased sample covariance with divisor $T-1$, a RiskMetrics-style EWMA recursion filtered *about zero* [J.P. Morgan/Reuters, 1996], and the single-target linear shrinkage of Ledoit and Wolf [2004], $\hat{\delta}\hat{\Sigma} + (1-\hat{\delta})\mu I$ with divisor T and closed-form intensity $\hat{\delta}$ [Ledoit and Wolf, 2004]. The choice of zero as the EWMA centre is asserted in an oracle test as a convention bridge rather than derived, because the two definitions disagree whenever the mean is material (`docs/limitations.md`, item 39).

2.5 Path-dependent payoffs

For M equally spaced fixings with $h = T/M$, the discretely monitored geometric average G satisfies $\ln G \sim N(\mu_G, s_G^2)$ with

$$\mu_G = \ln S + (r - q - \tfrac{1}{2}\sigma^2) h \tfrac{M+1}{2}, \qquad s_G^2 = \sigma^2 h \frac{(M+1)(2M+1)}{6M} \longrightarrow \frac{\sigma^2 T}{3}, \tag{9}$$

whose $M \to \infty$ limit is the textbook continuous-averaging variance and serves as the internal check on the derivation (`docs/model_cards/path_dependent_options.md`). Barriers are monitored discretely on the simulation grid, with the continuity correction of Broadie et al. [1997], $B_{\text{eff}} = B \exp(\mp \beta \sigma \sqrt{dt})$ with $\beta = -\zeta(1/2)/\sqrt{2\pi}$, available as an option (`docs/model_cards/path_dependent_options.md`).

3 Software Architecture

3.1 Layers and permitted direction of dependency

The system is layered so that numerics live in exactly one place. The C++20 core in namespace `quantrisk` owns all mathematics, in the directories `core`, `math`, `stochastic`, `pricing`, `monte_carlo`, `risk`, `portfolio` and `stress`; bindings expose it, a Python package gives each submodule a face and adds five facade classes (`docs/architecture.md` §1–§2, §5). The permitted dependency arrows are frozen in `docs/architecture.md` §3:

$$\text{experiments} \to \text{python API} \to \text{bindings} \to \text{C++ core} \to \text{Eigen},$$

with tests pointing at the core and the API, benchmarks pointing at both plus the oracles, and the data layer pointing at the Python API only. Two prohibitions carry the design: oracles never become dependencies of the core, and data never becomes a dependency of tests. Each layer has an explicit "must NOT" column — the core may not touch the network, hold global RNG state or duplicate Eigen's linear algebra; the bindings may not reimplement formulas or add validation logic; the Python layer may not shadow C++ numerics with a second implementation (`docs/architecture.md` §2).

3.2 The Python face, and how it is prevented from computing anything

Because `_quantrisk` binds its submodules as attributes, `sys.modules` is never populated for them and `fromquantrisk.riskimportX` cannot resolve; Phase 9 gave each of the eight Python-visible submodules — `stats`, `special`, `stochastic`, `monte_carlo`, `pricing`, `risk`, `portfolio` and `stress` — a small Python file re-exporting the extension's public names, and for four of them the facade class (`docs/architecture.md` §2; `docs/phase_reports/phase-09-research-api.md`). The claim that these files contain no mathematics is not asserted in prose but checked structurally: an AST walk rejects binary arithmetic, comparisons, `for` and `while` nodes in every shim, and every callable's `__module__` must be the extension (`docs/phase_reports/phase-09-research-api.md` §6).

That guard replaced a weaker one, and the substitution is worth recording because it is the kind of change that usually degrades a suite. The old test asserted that `quantrisk.stats.__name__` started with `quantrisk._quantrisk` and that `__file__` was not a `.py` file. Phase 9's design makes both conditions false, so the test failed. Rather than delete it, the assertion was moved to the invariant the old one stood in for, and the new version was checked for falsifiability: appending `mean_of_sample=sum([1,2])/2` to the shim makes the new test fail, whereas the old `__name__` check would have passed that edit (`docs/phase_reports/phase-09-research-api.md` §6). Two further guards are intentional: each face exposes `module.__core__` so a test can prove a value came from C++, and `importquantrisk` does *not* import `quantrisk.data`, verified in a fresh interpreter by printing `sys.modules` membership rather than by inspecting the current process (`docs/architecture.md` §2; `docs/phase_reports/phase-09-research-api.md`).

3.3 Interface freeze

Public surface is frozen phase by phase and later changes require a written migration note. Phase 1 froze the namespace, the numeric type aliases (`Real=double`, `Time`, `Rate`, `Volatility`, `Money`, `Count`, `Seed`), the `Rng` abstraction, the normal distribution functions, the statistics primitives, `version()`, `build_metadata()` and `ValidationError`; Phase 2 froze the instrument and result types; `docs/project_scope.md` §10 lists the frozen surface of each subsequent phase (`docs/architecture.md` §6; `docs/phase_reports/phase-01-engineering-foundation.md`).

3.4 Build, packaging and dependency policy

One CMake definition builds the static core, the pybind11 module, a reference-value dumping tool and the Catch2/CTest suite, with `dev`, `release` and `ci` presets, and `scikit-build-core` [the scikit-build maintainers, 2024] makes `uvpipinstall-e.` build the same tree (`docs/phase_reports/phase-01-engineering-foundation.md`). Runtime dependencies are numpy and pandas only; the oracle set (scipy, statsmodels, QuantLib, PyPortfolioOpt, cvxpy, scikit-learn) sits in an optional group and matplotlib in a `plot` group (`pyproject.toml`). Third-party C++ dependencies are pinned and fetched at configure time: Catch2 v3.8.1, pybind11 v2.13.6 [Jakob et al., 2024], and Eigen 3.4, introduced only when Phase 6 first needs dense linear algebra (`CMakeLists.txt`; `docs/limitations.md`, item 8). Everything is free software: `LICENSE` is MIT, and each phase gate records a line for "nothing paid for" (`docs/phase_reports/phase-06-portfolio-optimisation.md`).

A build-time network touch is one of the honest weaknesses: a fresh build directory fetches Catch2 and pybind11, and there is no vendored fallback (`docs/limitations.md`, item 5). Tests, benchmarks and experiments are offline, and the data layer is fixture-backed so that a missing key or connection cannot fail a test (`docs/validation_protocol.md` §6).

3.5 Foundational numerics

Randomness is confined to an instance-owned `std::mt19937_64` [International Organization for Standardization, 2020, Matsumoto and Nishimura, 1998]: no global RNG state, every stochastic entry point takes an explicit seed, `uniform01` returns values in $[0, 1)$, and `uniform_index` is unbiased by rejection over the full 2^{64} range. Normal variates come from the project's own Marsaglia polar implementation with a cached partner rather than from `std::normal_distribution`, whose output is implementation-defined across standard library implementations — a choice made explicitly so that bit-identical replay depends on (seed, N, code version) and on the platform's mathematical library, not on which `<random>` the compiler ships (`docs/phase_reports/phase-01-engineering-foundation.md` §2). Φ is computed as $0.5\,\mathrm{erfc}(-x/\sqrt{2})$ to keep relative accuracy in the left tail; the inverse is a rational approximation refined by one Halley step, evaluated on the lower half and reflected (`docs/phase_reports/phase-01-engineering-foundation.md`). Statistics use compensated summation, and the autocorrelation uses the Wallis denominator because that is the normalisation Christoffersen's statistic is stated with (`docs/phase_reports/phase-01-engineering-foundation.md` §2). Special functions (Lanczos $\log\Gamma$ with reflection, regularised incomplete gamma in both directions, chi-square survival function and its inverse) were added in Phase 5 because the tests needed them, and are cross-checked against SciPy live (`docs/phase_reports/phase-05-market-risk.md`).

3.6 Optional data layer

`quantrisk.data` is Python-only, uses the standard library and so adds no dependency. It provides SEC EDGAR, FRED/ALFRED and CFTC adapters behind a transport that rate-limits per host across processes, caches by URL, and refuses to send an anonymous request to a SEC host rather than sending one; provenance is an immutable record of source, download timestamp, series identifier and a SHA-256 digest taken over the bytes as served, and reading refuses a payload that no longer matches. Thirteen committed fixtures of real responses, totalling roughly 150 KB, back every test path, with the 2.4 MB CFTC archive kept as a labelled excerpt, and a socket-blocking test drives all three adapters with the sockets replaced by raisers, so the offline requirement is executed rather than asserted (`docs/phase_reports/phase-08-public-data.md`). No number in this report uses this layer (`docs/limitations.md`, item 49).

4 Pricing Models

4.1 Black–Scholes–Merton

The implementation follows Section 2 and returns not only a price but d_1, d_2, the method used and a `note` field, with a `validate()` that collects *every* violated constraint rather than

Table 1: Agreement with a live QuantLib 1.43 analytic engine, worst case over the recorded comparison grid, from `benchmarks/quantlib/results/pricing_vs_quantlib.json`. Absolute errors are in the natural units of each quantity; relative errors are evaluated only above the floors recorded in the same artifact.

Quantity	comparisons	worst absolute	worst relative
Black–Scholes price	2,464	1.49×10^{-13}	3.46×10^{-11}
Delta	—	7.97×10^{-15}	1.00×10^{-10}
Gamma	—	1.90×10^{-14}	3.17×10^{-13}
Vega	—	1.75×10^{-13}	3.29×10^{-13}
Theta	—	5.12×10^{-13}	2.53×10^{-10}
Rho	—	5.12×10^{-13}	1.07×10^{-7}

reporting the first (`docs/phase_reports/phase-02-deterministic-pricing.md`). Degenerate cases are handled rather than special-cased away: $T = 0$ collapses to intrinsic, $\sigma = 0$ to the discounted intrinsic of the forward, and the ATM-forward call equals the ATM-forward put.

Level-1 identities hold to floating-point noise. Worst put–call parity residual over the experiment grid is 8.00×10^{-15} on notionals ≤ 200 (`experiments/pricing_validation/results/summary.json`, key `worst_put_call_parity_residual`; `README.md`). Level-2 agreement against QuantLib's analytic European engine, over 2,464 price comparisons within an 18,816-row grid, is given in Table 1. Two points about that table are worth preserving: the relative figures are computed above a floor (`price_floor` $\approx 10^{-4}$, `greek_floor` 10^{-6}) recorded in the artifact, because relative error is meaningless near zero; and the Rho relative figure of 1.07×10^{-7} sits against an absolute error of 5.12×10^{-13}, which is the rounding scale of the quantity, not a disagreement.

4.2 Greeks by finite differences

Central-difference Greeks are computed under a documented `BumpPolicy`, and agreement with the analytic values is judged not against a tuned band but against a Richardson error estimate. The history of this check is a good illustration of the project's tolerance discipline. The first version asserted a 10^{-6} relative band; the observed residual for a 1% bump is 1.3×10^{-4}, which is the correct $O(h^2)$ truncation term — the bound had been guessed. It was replaced by an estimate of the truncation error rather than by loosening a number, and agreement holds for all five Greeks across seven regimes with the gap falling about fourfold when the bump halves (`docs/phase_reports/phase-02-deterministic-pricing.md` §6, §8). The same correction was applied later to the CLI's own check, whose guessed 5×10^{-5} gap bound failed at 6.39×10^{-5} and was replaced by a convergence-ratio assertion of 4.00 and 4.00 across two independent halvings (`docs/phase_reports/phase-09-research-api.md` §6).

4.3 The lattice

The Cox–Ross–Rubinstein tree is $O(dt)$ in $dt = T/N$ and no Richardson extrapolation is applied or claimed (`docs/limitations.md`, item 11). Convergence is presented as a rate,

Table 2: Fitted convergence order of the CRR lattice toward the Black–Scholes value. Source: `experiments/pricing_validation/results/summary.json`, key `crr_convergence_slope`, with relative error at $N = 3200$ from the same file's `final_lattice_relative_error`.

Regime	fitted slope	expected	slope s.e.	rel. error at $N = 3200$
atm_call	−0.9941	−1	0.0031	6.58×10^{-5}
otm_call	−1.0054	−1	0.1621	1.97×10^{-4}
high_vol_call	−0.9924	−1	0.0041	7.34×10^{-5}
short_dated_atm_put	−0.99967	−1	0.00016	8.31×10^{-5}

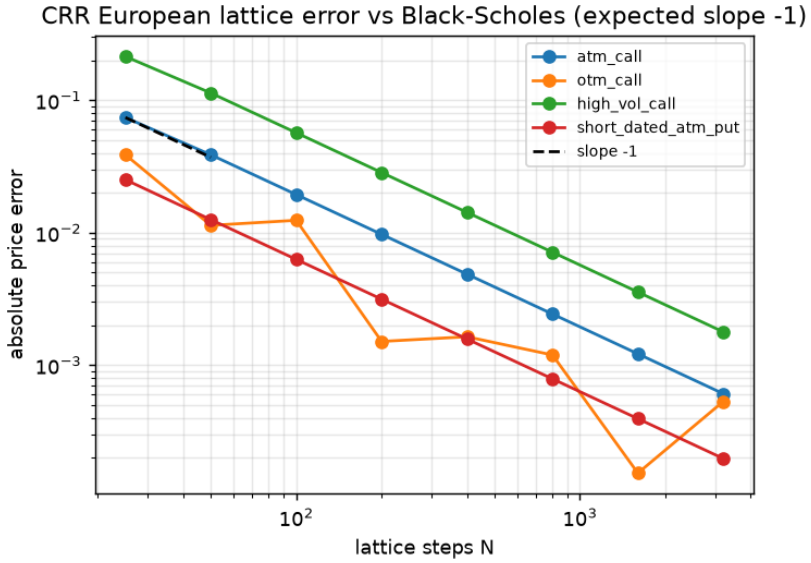


Figure 1: CRR lattice error against N on log-log axes, the source of Table 2. Generated figure, committed at `experiments/pricing_validation/results/lattice\_convergence.png`.

not as a point: a log-log least-squares fit of $\log|\text{error}|$ on $\log N$ per regime, with the standard error of the slope and the deviation from theory expressed in standard errors. Table 2 gives the result. The short-dated at-the-money put fits so tightly that its small absolute deviation exceeds two standard errors; the artifact's own `reading` field attributes that residual to the next term of the expansion rather than to a disagreement about the order, and says so rather than dropping the row (`experiments/pricing_validation/results/summary.json`).

Comparing two different lattices is a different exercise and is treated as one. Against QuantLib's `BinomialVanillaEngine(...,"crr",N)` — a different discretisation of the same dynamics — the absolute gap in the extreme corners of the grid reaches 1.82 at 50 steps, 0.457 at 200 steps and 0.114 at 800 steps (`benchmarks/quantlib/results/pricing_vs_quantlib.json`, `worst_absolute_error`). Those numbers are not errors of our implementation: at 100% volatility over five years on a coarse tree the gap is *smaller than each lattice's own error against Black–Scholes*, so the test asserts the normalised property rather than a raw threshold (`docs/limitations.md`, item 12). The summary key `lattice_error_ratio_*` is reported only as informative away from cancellation

Table 3: Path-dependent pricing against live oracles. The two barrier rows are the same estimator with and without the Broadie et al. [1997] correction, so their gap is the monitoring bias itself. Source: `benchmarks/quantlib/results/path_dependent_vs_quantlib.json`, key `worst_relative_error`.

Family	reference	worst rel. error
`asian_geometric_closed_form`	QuantLib analytic	5.55×10^{-4}
`asian_geometric_simulated`	QuantLib analytic	2.30×10^{-3}
`barrier_discrete`	continuous analytic	1.40×10^{-1}
`barrier_bgk_corrected`	continuous analytic	1.00×10^{-2}
`heston_degenerate` ($\xi = 0$)	Black–Scholes	2.09×10^{-3}
`heston_analytic`	`AnalyticHestonEngine`	4.47×10^{-3}

points, where the observed ratio spans 1.2×10^{-8} to 766 (`docs/limitations.md`, item 13; `benchmarks/quantlib/results/pricing_vs_quantlib.json`).

4.4 Asian and barrier options, and where discretisation actually costs

Arithmetic Asian options are priced by Monte Carlo with the geometric average of the *same* paths as control variate; geometric Asians are priced both by simulation and by the closed form of Section 2; barriers are up-and-out and down-and-out with discrete monitoring and the optional continuity correction. Table 3 collects the worst relative errors against four oracle families recorded in `benchmarks/quantlib/results/path_dependent_vs_quantlib.json`, on 25 rows with 200,000 paths each.

The 14.0% on the raw discrete barrier is not a bug but the economics of discrete monitoring: a barrier checked at M dates can only be crossed at those dates, so the knockout probability is lower and the option worth more than the same contract monitored continuously, and the correction reduces the gap by roughly fourteenfold (`docs/model_cards/path_dependent_options.md`; `docs/limitations.md`, item 23). The correction is an $O(\sqrt{dt})$ asymptotic result for a single barrier on a lognormal diffusion, not an exact treatment of continuous monitoring, and the monitoring grid is the simulation grid, so `steps` cannot be refined for one purpose without the other (`docs/limitations.md`, items 23–24). The residual 5.5×10^{-4} on the geometric Asian closed form is calendar-day rounding of the fixing schedule, because fixing and monitoring dates are year-fraction arithmetic with no business-day convention (`docs/limitations.md`, item 22).

Two defects in this area were found only because an oracle was consulted, and both are described in Section 9: the continuity correction was initially implemented moving the effective barrier the wrong way, with a test that asserted the wrong inequality and passed because the implementation shared the misconception; and a mismatched fixing schedule between the benchmark and the closed form produced a spurious 12% disagreement between two correct formulas (`docs/phase_reports/phase-04-path-dependent-heston.md` §6).

4.5 Heston simulation

The scheme is full-truncation Euler: the diffusion coefficient of both equations uses $\max(v, 0)$ and the variance update is clamped, with correlated normals built as Z_1 and $\rho Z_1 + \sqrt{1-\rho^2}\, Z_2'$ so one stream per path carries the correlation exactly. It is not an exact simulation, and every result carries a `note`, the clamp count in `negative_variances_clamped`, and a bias probe (`heston_step_refinement_gap`) (`docs/model_cards/heston.md`). Validation here is deliberately described as weaker than the Black–Scholes section, for a structural reason: the oracle is QuantLib's semi-analytic engine, itself a numerical quadrature, so the measured 4.5×10^{-3} combines our discretisation bias with the oracle's integration tolerance (`docs/limitations.md`, item 25). The only exactness claimed is the $\xi = 0$ collapse to Black–Scholes, measured at 2.1×10^{-3} at 200,000 paths over 250 steps and read as within sampling error (`docs/model_cards/heston.md`). Supporting checks in the same card: the risk-neutral martingale property $\mathbb{E}[S_T] = Se^{(r-q)T}$ inside four standard errors; positivity of every terminal and variance value over 20,000 paths including a Feller-violating parameter set; the sign of skew (an out-of-the-money put priced higher at $\rho = -0.9$ than at $\rho = +0.9$); and step refinement with differences inside four combined standard errors for 20, 80 and 320 steps. There are no Heston Greeks, no smile calibration, and no path-dependent payoff under Heston (`docs/limitations.md`, item 27).

5 Monte Carlo Methods

The estimator is $V_0 = e^{-rT}\mathbb{E}^{\mathbb{Q}}[\text{payoff}(S_T)]$ with $\widehat{V}_N = e^{-rT}N^{-1}\sum_i \text{payoff}_i$ and $\text{SE}(\widehat{V}_N) = s/\sqrt{N}$, s the sample standard deviation of the discounted payoffs (`docs/mathematical_specification.md` §5). The separation of bias from sampling error is load-bearing here: a European payoff drawn from the exact terminal transition has zero discretisation bias, so all error is sampling error, whereas path-dependent payoffs and the Heston scheme carry both. Each estimate reports price, standard error, confidence interval, path count, count of independent units, seed, sample variance, the fitted control-variate coefficient and wall-clock runtime (`docs/phase_reports/phase-03-monte-carlo.md`).

5.1 Convergence and standard-error calibration

Theory predicts $\widehat{V}_N - V_0 = O(N^{-1/2})$, and the claim is tested as a fitted slope of $\log|\text{error}|$ on $\log N$ over 10^3 to 3×10^6 paths. Three scenarios give -0.614 ± 0.089, -0.585 ± 0.096 and -0.507 ± 0.118 against the theoretical -0.5, all within 1.3 standard errors (`experiments/monte_carlo_convergence/results/summary.json`, key `convergence_slope_fit`; `README.md`).

A slope confirms the exponent; it does not confirm that the *reported* standard error is the right yardstick. Two further measurements address that. The first is coverage: over 12 combinations of seed count, path count and interval level, the empirical coverage of nominal 95 % and 99 % normal intervals fell inside the exact Binomial$(n, level)$ band in 12 of 12 cases (`experiments/monte_carlo_convergence/results/summary.json`, `coverage_intervals_inside_binomial_band`). The second is a z-score study: 492 rows comparing our estimate against QuantLib's analytic price over 4 scenarios × 3 methods × 40 seeds at 200,000 paths per run. With a correct estimator and a correctly reported standard

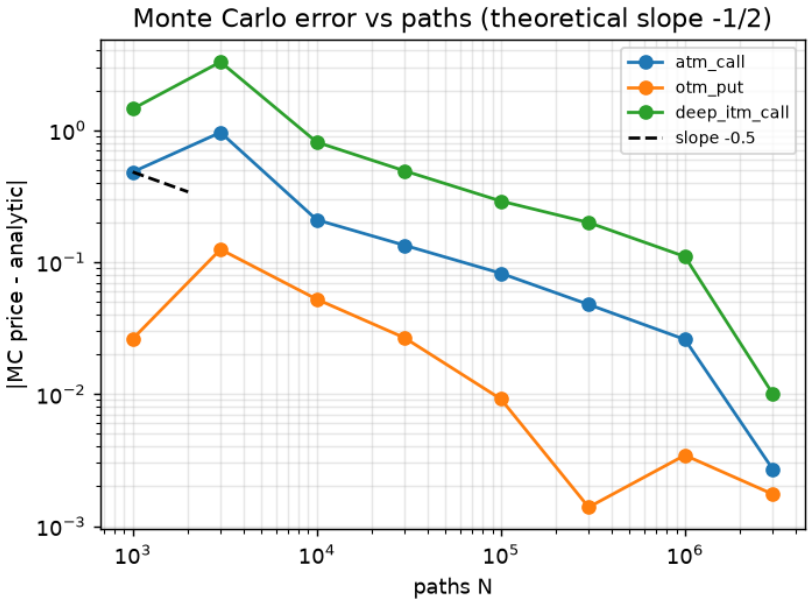


Figure 2: Fitted slope of $\log|\text{error}|$ on $\log N$ against the theoretical -0.5, the source of the figures in Section 5. Generated figure, committed at `experiments/monte_carlo_convergence/results/convergence\_slope.png`.

Table 4: Pooled z-scores of our Monte Carlo estimate against the live analytic oracle, 160 draws per method. Source: `benchmarks/quantlib/results/monte_carlo_validation.json`, key `z_score_vs_quantlib_analytic`.

Method	n	mean	std	range	within 2 SE
antithetic	160	-0.023	1.014	-2.04 to $+2.06$	98.8 %
control variate	160	$+0.050$	0.923	-2.11 to $+1.78$	98.1 %
plain	160	$+0.031$	0.930	-2.40 to $+2.10$	96.2 %

error, pooled z-scores should have mean near zero, standard deviation near one, and about 95 % inside ± 2; a systematically understated standard error shows up first as a standard deviation well above one. The measured values in Table 4 are consistent with all three (`benchmarks/quantlib/results/monte_carlo_validation.json`).

One negative result is recorded rather than dropped: QuantLib's own Monte Carlo engine could not be constructed in this build — `MCEuropeanEngine` rejected every traits string through the installed SWIG bindings — so the artifact carries `quantlib_mc_engine_used: false` with the reason, and the independent yardsticks that *are* used are the analytic price (the exact limit) and a NumPy/PCG64 simulation (`benchmarks/quantlib/results/monte_carlo_validation.json`; `docs/limitations.md`, item 20).

5.2 Variance reduction, measured out of sample

Antithetic pairs share one normal Z with the drift not negated, and satisfy the exactness identity $S^+S^- = S^2 \exp[2(r - q - \sigma^2/2)T]$ to 10^{-12} relative. Control variates use the

geometric Asian average for arithmetic Asian payoffs, and an in-sample adaptive coefficient. Because that coefficient is fitted on the same sample it is used to reduce, the engine's own reported variance is mildly optimistic, and the project's answer is to measure realised mean squared error out of sample over seed ensembles rather than to quote the in-sample number (`docs/limitations.md`, item 19). Over 40 seeds × 3 path budgets × 4 scenarios, the realised out-of-sample MSE reduction relative to plain Monte Carlo ranges from 1.12× to 2.77× for antithetic sampling and from 1.93× to 40.47× for control variates, with the largest gain on the deep-in-the-money long-maturity call and the smallest on the short-dated out-of-the-money call (`experiments/variance_reduction/results/summary.json`, key `mse_reduction_vs_plain`; `docs/phase_reports/phase-03-monte-carlo.md`).

Two Level-1 identities guard against a variance-reduction estimator that is merely *popular*: a payoff affine in the control must reproduce the analytic price exactly, and a constant payoff must return the discount factor. The deep-in-the-money call with strike 1.0 is affine in the control, and the control-variate estimator returns the analytic price to about 10^{-12} with a standard error below 10^{-10} and fitted coefficient of 1 (`docs/phase_reports/phase-03-monte-carlo.md` §6; `docs/model_cards/monte_carlo_gbm.md`). No quasi-Monte Carlo, Brownian bridge, stratification or moment matching is implemented, and the engine is single-threaded (`docs/limitations.md`, item 16).

6 Market Risk Estimation

Six point estimators, two bootstrap designs and three coverage tests were implemented in Phase 5, and the phase's research question was statistical rather than numerical: are the estimators and tests valid on synthetic distributions whose truth is known (`docs/phase_reports/phase-05-market-risk.md`). The design of that experiment is worth describing before its results, since it is what licenses the interpretations below. Data are generated as Gaussian with $\sigma = 2\%$, a Student-t with 3 degrees of freedom scaled to the same unconditional standard deviation, and a GARCH(1,1) process with $\alpha = 0.10$, $\beta = 0.85$ and ω chosen to match the same unconditional standard deviation [Bollerslev, 1986]; a fourth arm generates clustered violations from a two-state Markov chain with stationary rate 5 % and Pr(violate | violated) = 0.25, so the unconditional count is on target while the timing is clustered. Estimator bias uses 400 replications; realised violation rates use 200 independent calibration blocks of 10,000 observations scored over 20,000 out-of-sample days each; coverage-test size uses 2,000 replications of 250 days; bootstrap coverage uses 400 replications at $n = 500$ with 1,500 resamples against an unconditional target computed from 2,000,000 draws. All streams are seeded (`estimator7000+r`, `backtest100000+r`, and so on) so reruns reproduce bit for bit (`experiments/var_backtesting/results/var_backtesting_validation.json`).

6.1 Point estimators against known population values

Table 5 reports the outcome that the layer is most careful about: the Gaussian fit on fat-tailed data is wrong in *opposite directions* at the two confidence levels, because normal and t quantile spacings differ. The empirical estimator tracks nominal at both levels.

At $n = 250$ on Gaussian data the bias of the Gaussian estimator at 95 % is -8.2×10^{-5}, or 0.25% relative (`experiments/var_backtesting/results/var_backtesting_validation.`

Table 5: Out-of-sample realised violation rates (200 blocks × 20,000 days) against population VaR levels. Read the two $t(3)$ rows together: a normal fit is not conservatively wrong, it is wrong in opposite directions. Source: `experiments/var_backtesting/results/estimator_convergence.csv`, block `A_out_of_sample`; headline values also in `README.md`.

Data	Level	Population VaR	nominal rate	empirical realised	Gaussian realised
Gaussian $\sigma = 2\%$	95 %	0.03290	5 %	5.034 %	5.021 %
Gaussian $\sigma = 2\%$	99 %	0.04653	1 %	1.014 %	1.009 %
$t(3)$, same σ	95 %	0.02717	5 %	5.014 %	**3.291** %
$t(3)$, same σ	99 %	0.05243	1 %	1.021 %	**1.389** %

Table 6: Rejection rates of the three coverage statistics, 2,000 replications of 250 days at a 5 % nominal cutoff. The last row is the discriminating result: an unconditional-frequency test accepts a clustered model most of the time. Source: `experiments/var_backtesting/results/coverage_test_size_power.csv`, block `B_size_power`.

Arm (true violation rate)	Kupiec	Independence	Conditional
correctly calibrated iid (0.05)	0.0505	0.0235	0.0380
over-calibrated iid (0.02)	0.7525	0.0206	0.6136
under-set VaR (0.12)	0.9860	0.0600	0.9620
clustered, correct marginal (0.05)	0.1200	0.5540	0.5345

`json`, `headline.gaussian_95_var_bias_at_n250`). The layer's own reading of the two $t(3)$ rows is that any explanation promising a one-sided safety margin from a normal fit is false (`docs/model_cards/var_es.md`), and the estimators are retained as the reference case, not as a defensible production choice (`docs/limitations.md`, item 30). Empirical 99 % VaR at 250 observations averages two or three order statistics, and `quantile_standard_error` returns NaN rather than a plausible zero when the loss distribution has a flat spot at that level — common for option P&L, where most paths finish worthless (`docs/limitations.md`, item 29).

6.2 Coverage tests: size, power and what each can see

Table 6 gives the four calibration arms. Three readings matter. First, the Kupiec test has its nominal size — 5.05% rejection at the 5 % level, with exact binomial interval $[4.13\%, 6.10\%]$ — and is conservative at a 1 % cutoff (0.0075) because the statistic is a function of an integer count. Second, power against an under-set level reaches 1.000 by 500 days. Third, and most useful: with clustered violations at the *correct* marginal frequency, Kupiec rejects 12.0% of samples while the independence test rejects 55.4%, and the converse holds too — with a pure frequency error the independence test rejects only 5 to 6.8% of the time, so it is not manufacturing a clustering signal (`experiments/var_backtesting/results/coverage_test_size_power.csv`; `docs/phase_reports/phase-05-market-risk.md` §6).

These tests are marginal-frequency tests on a *given* VaR series. They cannot validate the model that produced the series, they have no power against an error that preserves both the violation rate and its timing, and no multiple-comparison correction is applied across the three statistics (`docs/limitations.md`, item 34). The implementation reports

Table 7: Bootstrap coverage of the true unconditional VaR. Nominal level 90 %; intervals are exact binomial. Source: `experiments/var_backtesting/results/bootstrap_coverage.csv`.

Data	Design (block length)	Coverage	95 % interval
iid Gaussian	iid	0.8725	[0.8358, 0.9036]
iid Gaussian	moving block (8)	0.8700	[0.8330, 0.9014]
GARCH(1,1) clustered	iid	0.6250	[0.5755, 0.6726]
GARCH(1,1) clustered	moving block (8)	0.6925	[0.6447, 0.7374]

a `degenerate` flag and NaN rather than a p-value of 1 when a state has no transitions, so that a null result cannot be read as a clean bill of health (`docs/model_cards/var_es.md`).

6.3 Uncertainty by resampling

Percentile bootstrap intervals for the 95 % VaR, at a nominal 90 % level, $n = 500$, 400 replications, are given in Table 7. On iid data the percentile interval is slightly under nominal and the block design costs nothing. Under GARCH clustering the iid design loses almost 28 percentage points of coverage; the block design recovers about 7 of them, and the exact intervals do not overlap, so the improvement is real — and the block design still falls roughly 21 points short of nominal (`experiments/var_backtesting/results/bootstrap_coverage.csv`; `docs/limitations.md`, items 31–32). This is reported as a limitation rather than fixed by tuning the block length, and the default round($n^{1/3}$) rule is explicitly flagged as a general-purpose choice that was never swept against coverage (`docs/phase_reports/phase-05-market-risk.md` §8).

No Cornish–Fisher expansion, extreme-value tail fit, filtered historical simulation or expected-shortfall backtest of the Bellini–Frittelli or Acerbi–Tasche style named in the register is implemented, and the estimand here is the unconditional quantile, not a daily re-estimated conditional one (`docs/limitations.md`, items 33, 35).

6.4 Do the conclusions survive real data?

Everything above is measured on processes whose truth is known, which is what makes a bias measurable and also what keeps each conclusion one step from an empirical claim. The study in `experiments/real_data_risk_study/` re-runs the same estimators and the same tests on real daily market observations: three FRED series used as risk factors — $DGS10$, the 10-year Treasury par yield; $T10YIE$, the 10-year breakeven inflation rate; $VIXCLS$, the CBOE volatility index — 3,177 aligned observations spanning 2014-01-03 to 2026-09-22, against an assumed book of fixed notional exposures. The factor moves are observations; the exposures are assumptions; no number describes a trade. Each VaR is calibrated on the 250 observations *preceding* the return it scores, stepping by 5, giving 586 scored days (Table 8).

Three results, one of them negative. **The fat-tail prediction reproduces**: the synthetic $t(3)$ arm of §6 forecast that a Gaussian 99 % VaR would over-reject, and on data nobody generated it realised 2.048 % against 1 % nominal with an interval that excludes the nominal rate, while historical simulation at 1.365 % covers it. **The coverage tests do not discrim-**

Table 8: Out-of-sample violation rates on 586 real daily observations, trailing 250-day calibration, with exact binomial intervals. Source: the committed run of `experiments/real_data_risk_study/results/real_data_risk_study.json`. The Gaussian 99 % interval excludes the nominal rate; that is the finding.

Estimator	Level	Nominal	Realised	Exact 95 % interval
Historical	95 %	0.05000	0.06314	[0.04484, 0.08598]
Gaussian	95 %	0.05000	0.05631	[0.03908, 0.07818]
Historical	99 %	0.01000	0.01365	[0.00591, 0.02672]
Gaussian	99 %	0.01000	**0.02048**	[**0.01062**, **0.03550**]

inate on this sample: Kupiec $p = 0.160$, Christoffersen independence 0.287, conditional coverage 0.211 — none rejects, and a runs test gives $z = -1.17$ (67 runs against 70.33 expected, `experiments/real_data_risk_study/results/real_data_risk_study.json`), the right sign with no power. The machinery was shown to reject 55.4 % of clustered synthetic series, so this is evidence about the sample, not against the tests, and it is published rather than dropped. **The truth-free version of the bootstrap question agrees**: the moving-block standard error is 1.072× the iid one, a mild 7 % understatement consistent with the weak clustering.

The covariance conclusion does not survive, and the way it fails is instructive. Rolling minimum-variance books from each estimator, scored on realised portfolio variance over 582 windows (`experiments/real_data_risk_study/results/real_data_risk_study.json`), rank *ewma* < *sample* < *shrinkage*. The synthetic study of §7 ranked the same three by mean forward-variance ratio as shrinkage 1.169 < sample 1.183 < ewma 1.241: the two ends exchange places while the middle holds. Shrinkage wins where a short window makes the sample covariance *unstable*, which is the regime the synthetic process was built to create; over a twelve-year record of three persistent series the structured target is what lags. Neither number is wrong. An estimator ranking is a property of the process it was measured on, which is the reason §7 reports the ratio against an informed solver rather than a bare winner.

Three questions are refused inside the artifact rather than proxied. The cost of estimating covariance against an informed solver needs an observable true forward covariance, which real data does not have. Bootstrap *coverage* needs a truth independent of the series being resampled; the only candidate is the full-sample VaR, and intervals built by resampling that series cover it by construction — an earlier version measured exactly 1.000 this way and the figure was discarded and named as discarded. Whether the book is profitable needs a return, which it does not have (`docs/limitations.md`, items 61–62; `docs/phase_reports/phase-11-real-data-risk-study.md`).

7 Portfolio Optimization

The portfolio layer contains three covariance estimators and five solvers, 2,695 lines of header and source across the layer including bindings (`docs/phase_reports/phase-06-portfolio-optimisation.md` §3). Two properties distinguish it from a typical implementation exercise.

The first is that *every solution carries a certificate*. The active-set solver treats each

Table 9: Level-2 agreement on 75 problems. "Weights" is the largest coordinate difference against the named oracle; "objective" is the largest relative gap in the optimised quantity. Risk parity has no PyPortfolioOpt implementation in version 1.6.0 and CVaR is solved by PyPortfolioOpt through cvxpy, so those columns are labelled duplicates rather than two independent references. Source: `benchmarks/pyportfolioopt/results/optimisation_vs_oracles.json`, key `agreement.per_problem`.

Problem	weights vs PyPortfolioOpt	weights vs cvxpy	worst relative objective gap
`min_variance`	3.5×10^{-13}	2.4×10^{-12}	7.3×10^{-16}
`efficient_return`	4.0×10^{-13}	2.4×10^{-12}	6.5×10^{-14}
`max_sharpe`	7.0×10^{-9}	7.0×10^{-9}	1.2×10^{-11}
`risk_parity`	1.4×10^{-11}	1.4×10^{-11}	2.8×10^{-12}
`minimise_cvar`	4.7×10^{-7}	4.7×10^{-7}	4.5×10^{-9}

candidate support as a saddle-point KKT system rather than forming Σ^{-1}, and verifies the answer against the original data for primal feasibility, dual feasibility and complementary slackness; `verified_optimal` is therefore a proof, not a convergence message (`docs/model_cards/portfolio_covariance_and_optimisation.md`). The choice is load-bearing: with a flat or duplicated asset, the normal equations return the minimum-*norm* stationary point rather than the optimum — on $\Sigma = \mathrm{diag}(0.04, 0)$ they put the whole position in the risky asset and report variance 0.04, where the saddle system returns the true answer of holding the flat asset at zero variance, and that case is a test rather than a footnote. Likewise, ill-conditioned inputs are refused rather than repaired: the guarded `LLT` solve returns NaN with a reason when the matrix is not positive definite or when rcond $< 10^{-12}$, and every estimate carries its spectrum, `positive_semidefinite` flag and note (`docs/model_cards/portfolio_covariance_and_optimisation.md`).

The second is that the layer's headline result is about the *input*, not the solver.

7.1 Agreement with independent solvers

Table 9 reports the 75-problem benchmark (5 asset counts × 3 seeds × 5 problem families) against PyPortfolioOpt's SLSQP route and cvxpy with OSQP for the quadratic programmes and SCS for the conic ones. Weights and objectives are reported separately and deliberately, because on degenerate problems they carry different meanings: an LP optimum may be an entire face, so the vertex each method reports is a property of the algorithm, while the objective is the comparable quantity (`docs/model_cards/portfolio_covariance_and_optimisation.md`).

Constraints were checked on all 75 solutions independently of the agreement figures: the worst budget residual is 1.4×10^{-12} and the worst long-only violation is exactly 0 (`benchmarks/pyportfolioopt/results/optimisation_vs_oracles.json`; `README.md`). The estimators themselves were compared separately, in tests rather than in the benchmark: sample covariance agrees with `numpy.cov(ddof=1)` to 7.2×10^{-16} relative, and the Ledoit–Wolf shrinkage estimate agrees with scikit-learn's `LedoitWolf` to 7×10^{-19} absolute (`tests/python/test_covariance_vs_oracles.py`; `README.md`).

Two findings in this comparison came from checking, not from assuming, and both are

Table 10: Forward variance ratio against an informed solver, by estimation window and estimator. Values above 1 mean the estimated matrix costs extra variance. Source: `experiments/portfolio_optimization/results/portfolio_optimisation_study.json`, key `estimation_cost.by_window_and_estimator`.

Window	sample	EWMA	shrinkage
40	1.2176	1.2560	**1.1686**
60	1.1626	1.2287	**1.1463**
125	**1.1326**	1.2334	1.1331
250	1.2194	1.2453	1.2265

recorded in Section 9: the first draft of the benchmark measured the rounding of PyPortfolioOpt's `clean_weights()` rather than the optimisation, and the max-Sharpe row is loose on our side — both oracles agree with each other to about 10^{-10} while each sits at 7×10^{-9} from us, which a full-precision in-process probe resolved by showing the achieved Sharpe ratio matching to $\leq 1.3 \times 10^{-12}$ absolute, so the weight gap is the flat frontier rather than a wrong answer (`docs/phase_reports/phase-06-portfolio-optimisation.md` §6).

7.2 What estimating a covariance costs

Numerical agreement of order 10^{-9} is not the interesting quantity, because the matrix being inverted is an estimate. A rolling-origin study on a synthetic two-regime factor process — 8 assets, 600 monthly observations, a regime break at observation 300 where both factor volatilities double and loadings shift by 0.35, a 20-day forward horizon stepped by 5, EWMA $\lambda = 0.94$ — scores 1,119 frozen portfolios against a forward covariance known in closed form, over 373 solved windows, and reports the ratio of each frozen portfolio's forward variance to that of the portfolio an informed solver would have held (`experiments/portfolio_optimization/results/portfolio_optimisation_study.json`, key `data`). Table 10 gives the result.

Six readings come out of this artifact, each quoted with the figure that supports it. (a) Estimation, not optimisation, is the dominant error: the smallest measured penalty is about 13% in forward variance (best cell 1.1326) against a worst numerical objective disagreement of 4.5×10^{-9}, roughly seven and a half orders of magnitude (`experiments/portfolio_optimization/results/portfolio_optimisation_study.json`, `findings.estimation_is_the_dominant_cost`). (b) Shrinkage wins where instability bites: at window 40 it lowers the ratio from 1.2176 to 1.1686, cuts turnover from 0.2067 to 0.1381, and raises effective asset count from 5.10 to 5.97. (c) More history is not monotonically better: window 250 is 0.087 *worse* than 125 for the sample estimator, because it pools two different joint distributions. (d) Non-stationarity dominates estimation error: forward slices straddling the regime break cost 1.57 to 1.60 for every estimator, against 1.075 to 1.222 inside a single regime. (e) The refusal is the point: on a 5-observation window of 8 assets the sample matrix has condition number 6.7×10^{17} and the solve refuses, while the shrunk matrix at condition number 8.0 solves and costs 0.0086 rather than 0.0111 in forward volatility. (f) An optimiser is only as honest as its input: with an asset flat in the sample but volatile in the truth, the solver correctly concentrates into it (effective_assets = 1.00) and carries 3.5× the forward volatility of the well-conditioned

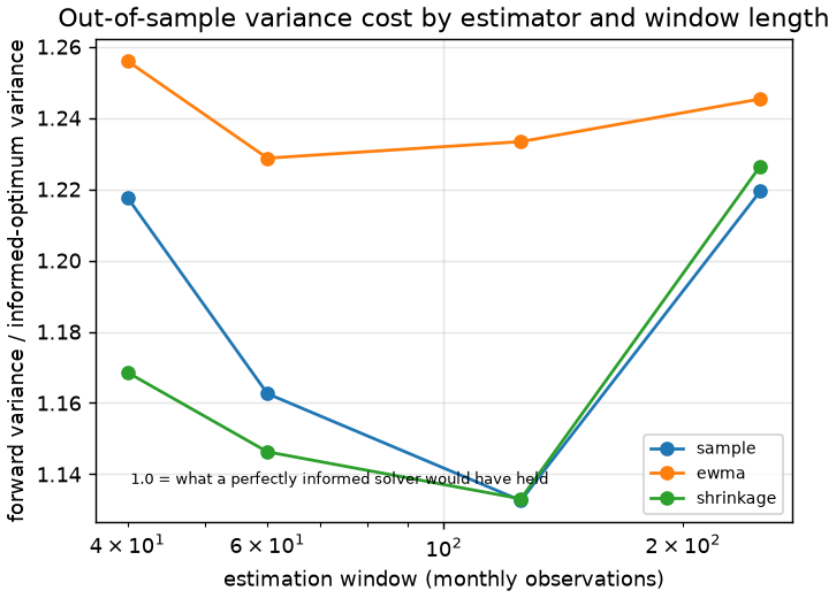


Figure 3: Forward variance ratio against an informed solver by window and estimator, the source of Table 10. Generated figure, committed at `experiments/portfolio\_optimization/results/estimator\_cost\_by\_window.png`.

case.

A fourth objective's behaviour completes the argument. Comparing all four objectives fitted on the same shrinkage-estimated windows and scored on the same forward slice, max-Sharpe — the only one that consumes an estimated *mean* — is worst on every forward metric, with forward volatility 0.01856 against 0.01166 for minimum variance and realised worst loss 0.02691 against 0.01441 (`experiments/portfolio_optimization/results/objective_comparison.csv`; `experiments/portfolio_optimization/results/portfolio_optimisation_study.json`, key `objective_comparison`). This is error maximisation measured rather than asserted, and it is why the project records that no expected-return model exists in this codebase and that any use treating a sample mean as skill inherits the consequence (`docs/limitations.md`, item 37).

Robustness cases were run alongside: near-collinear twins give condition numbers of 4.2×10^8 (sample) and 6.3×10^8 (EWMA), with the solve still succeeding and the estimate's note reporting the instability, against 56.9 for the shrunk matrix on the same input (`experiments/portfolio_optimization/results/solver_robustness.csv`). With every asset below the risk-free rate the maximum Sharpe ratio is negative and attained at a corner rather than a tangency, and the search certifies the least-bad portfolio; with an unreachable target the solution reports `feasible=false` with the size of the bound violation and a note naming the violated bounds, because such a target still has an equality solution — it simply requires shorting (`docs/model_cards/portfolio_covariance_and_optimisation.md`).

The scope is narrow and stated as such: long-only, fully invested, single-period, with no position caps, sector or turnover constraints, no transaction cost netted against return, an EWMA filtered about zero, and a shrinkage target of the scaled identity rather than a diagonal or custom target (`docs/limitations.md`, items 36–40). The simplex is dense,

two-phase, uses Bland's rule, and is exponential in the worst case; it was validated at ≤ 12 assets and 250 scenarios (`benchmarks/pyportfolioopt/results/optimisation_vs_oracles.json`) and is not a production LP (`docs/limitations.md`, item 41).

8 Stress Testing

The stress layer, 1,152 lines across four headers and four sources, answers a different question from the pricing or optimisation code: given a book described by factor exposures and a scenario described as data, what happens to the risk, and can the answer be attributed to its causes (`docs/phase_reports/phase-07-stress-testing.md` §1)? It maps exposures and never re-prices instruments. The mapping is deterministic,

$$\Delta\text{P\&L} = \sum_i \delta_i s_i + \gamma_i s_i^2 + d_i a_i + v_i a_i + c_i a, \tag{10}$$

where γ is quoted with the one-half absorbed — stated because the Black–Scholes gamma a caller would derive it from does not carry it, and the revaluation test depends on the convention. Portfolio dispersion uses the first-order beta δ + duration + vega + credit, because convexity moves the mean rather than the variance and folding a squared term into a covariance would make the second moment depend on the scenario's direction (`docs/phase_reports/phase-07-stress-testing.md` §2).

Factor classes decide what a shock means: equity factors are shocked relatively, rate, volatility and credit factors absolutely, both units are resolved wherever the quoted level permits, and a shock that states both and disagrees is rejected. A distinction that survives aggregation is that an absent sensitivity is not a zero: an empty `vega` block means no vega sensitivity was recorded, which is a weaker and more honest claim than vega being zero, and it is what forces `beta_of` to take the factor count rather than sizing itself from whichever block happens to be first (`docs/model_cards/scenario_stress_testing.md`; `cpp/src/stress/engine.cpp`).

8.1 Closure, independent numerics, and the measured truncation error

Attribution is exact by construction, so its residuals are a bug report rather than a tolerance. Across 28 scenario/book combinations the worst factor-attribution residual, position-attribution residual and level/dispersion decomposition residual are each exactly 0.0 (`experiments/stress_testing/results/stress_testing_study.json`, key `attribution_closure`; `experiments/stress_testing/results/scenario_ranking.csv`). The level/dispersion split of ΔVaR is exact only because the Gaussian quantile is affine in (mean, sigma); the code comment says so, and the limitation says the split would not telescope for a historical or Cornish–Fisher estimator (`docs/limitations.md`, item 45). Independent recomputation in NumPy of the mapping, covariance shift, historical replay and its quantile agrees to 10^{-9} relative, stressed VaR against SciPy's own normal quantile to 10^{-9}, and Euler component VaR against a central-difference gradient of the VaR formula to 3×10^{-10} against a bound set at 10^{-6} — about 3,000-fold inside its tolerance, which is what makes the check discriminate rather than accommodate (`tests/python/test_stress_vs_oracles.py`; `docs/phase_reports/phase-07-stress-testing.md` §6).

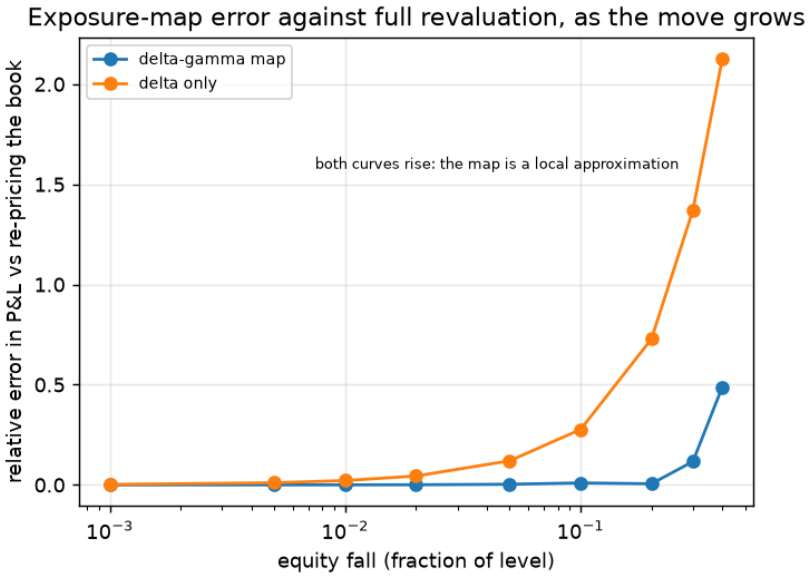


Figure 4: Relative P&L error of the delta–gamma and delta-only maps against a full Black–Scholes revaluation, the source of Table 11; the non-monotone segment is visible. Generated figure, committed at `experiments/stress\_testing/results/linearisation\_error.png`.

The linearisation error is then measured against the thing it replaces: a three-strike call book shocked through the exposure map and re-priced through the Phase 2 Black–Scholes engine.

Convexity buys two orders of magnitude at a 1 % move, and the delta-only map is already wrong by more than a fifth of the P&L there. Between 10 % and 20 % the delta–gamma error *improves*, because the cubic term changes sign through the strike region and partially cancels; the project's reading is that a bound read off one point of this curve would be wrong at another (`docs/limitations.md`, item 43). Monte Carlo stress VaR at 400,000 paths reproduces the parametric number within 1 %, with the longer run the closer one, and the sampler's realised dispersion recovers the supplied covariance to within 2 %, cross-checked against NumPy's generator — a different RNG and a different normal transform, so a reproducibility defect in our path could not agree with it by construction (`docs/phase_reports/phase-07-stress-testing.md` §6).

8.2 The finding: stress inverts the safety ranking

Four books were built by the Phase 6 optimisers — equal weight, minimum variance, risk parity, minimum CVaR — on a synthetic 10-factor fixture (six equities, one rate, one volatility, one credit spread) with notionals of 100,000,000 and 95 % confidence, then ranked by base parametric VaR and re-ranked under each of seven named scenarios. Table 12 summarises.

No book changes its absolute risk rank in any of the seven scenarios, and the report is careful to say why that is not reassuring: the books started 20.8% apart in base VaR (base_var_spread = 0.2077) while the largest stress-induced multiplier spread was 16.8%, so the stability is a near miss rather than a property of the optimisers (`docs/limitations.`

Table 11: Truncation error of the exposure map against a full Black–Scholes revaluation of the same book, as relative error in P&L. The delta–gamma column is *not* monotone in the size of the move, which is why no single validity range is claimed. Source: `experiments/stress_testing/results/linearisation_error.csv`; `experiments/stress_testing/results/stress_testing_study.json`, key `linearisation`.

Equity fall	delta–gamma rel. error	delta only rel. error
0.1 %	1.35×10^{-6}	2.10×10^{-3}
0.5 %	3.36×10^{-5}	1.06×10^{-2}
1 %	1.33×10^{-4}	2.15×10^{-2}
2 %	5.22×10^{-4}	4.42×10^{-2}
5 %	3.02×10^{-3}	1.20×10^{-1}
10 %	9.75×10^{-3}	2.77×10^{-1}
20 %	5.37×10^{-3}	7.28×10^{-1}
30 %	1.16×10^{-1}	1.37
40 %	4.88×10^{-1}	2.13

Table 12: VaR multipliers by scenario, and how many of the four books change rank. "Level rank" is the absolute risk ranking by stressed VaR; "sensitivity rank" is the ranking by multiplier. Source: `experiments/stress_testing/results/stress_testing_study.json`, key `ranking`.

Scenario	max mult.	min mult.	spread	rank kept	most sensitive
`equity_-20pct`	6.113	5.234	0.168	0 of 4	`min_variance`
`risk_off`	5.606	4.934	0.136	0 of 4	`min_variance`
`vol_x2`	2.000	2.000	0.000	0 of 4	all equal
`correlation_+0.2`	1.367	1.297	0.054	0 of 4	`risk_parity`
`rates_+200bp`	1.010	1.005	0.005	0 of 4	`risk_parity`
`credit_+150bp`	1.0004	1.0002	0.0002	0 of 4	`risk_parity`
`rates_-100bp`	0.998	0.995	0.002	0 of 4	`min_cvar`

`md`, item 48). Ranked by multiplier instead of by level, the order inverts: under the equity crash all four books change rank, and `min_variance`, the safest book under normal dispersion, is the most stress-sensitive at 6.11×, while `equal_weight`, the riskiest normally, is the least at 5.23× (`experiments/stress_testing/results/stress_testing_study.json`; `README.md`). This is the Phase 6 result seen from the other end: optimising for an estimated covariance concentrates a book, and concentration is exactly what an equity stress amplifies. The `vol_x2` scenario gives every book a multiplier of exactly 2.000 with zero spread and a level share of exactly 0.0 — all of its VaR increase is dispersion, none of it loss already taken — which is the decomposition working as designed.

Beyond this, three scenario kinds exist (deterministic, historical replay, Monte Carlo); `assumptions` is a required field the engine reads and echoes, so a scenario cannot present a number without provenance; and a single-scenario result returns NaN for VaR and ES rather than relabelling one outcome as a risk measure (`docs/model_cards/scenario_stress_testing.md`). Missing capabilities are listed rather than implied: no higher-order terms,

Table 13: The sharp form, on the same three-call ladder as Table 11. $c = 6R/h^3$ must lie between the extremes of V''' along the segment; it does, at every shock tested. Source: the committed run of `experiments/linearisation_error_bound/results/linearisation_error_bound.json`.

Down move	R	$c = 6R/h^3$	V''' along the path	$\lvert R\rvert$/bound
0.1 %	$-1.149 \cdot 10^{-3}$	−6.8964	[−6.9224, −6.8180]	0.9962
1 %	−1.110	−6.6579	[−6.9224, −5.8358]	0.9618
5 %	−114.8	−5.5084	[−6.9224, −0.6521]	0.7957
10 %	−649.5	−3.8972	[−6.9224, +6.8807]	0.5630
20 %	−528.5	−0.3964	[−6.9224, 17.3234]	0.0229
30 %	+12,527	+2.7838	[−6.9224, 17.5449]	0.1587
40 %	+53,178	+4.9855	[−6.9224, 17.5449]	0.2842

Gaussian scenario moves only, one factor per asset class so a rate shock is parallel by construction and a butterfly or steepener is inexpressible, and no reverse stress testing — the engine answers "what does this shock do" and not "what shock would produce this loss", which is the direction most supervisory use of the word means (`docs/limitations.md`, items 44, 46, 47).

8.3 A remainder bound that predicts a sign change

The stress layer maps a shock to P&L with delta and gamma, `m(\delta)=\DeltaS\delta+\tfrac12\GammaS^2\delta^2`, and §8 measured how far that sits from a full re-pricing. The published account said the error grows with the shock; the published *data* refutes it, reaching 649.5 at a 10 % down move, 528.5 at 20 %, and 12,527 at 30 %. A trend sentence was the wrong shape for the object. The right object is a bound, and it predicts.

Proposition. *With $S, K, \sigma, T > 0$, $\delta > -1$, $h = S\delta$ and $R(\delta)$ the difference between the re-priced and mapped P&L, there is ξ strictly between S and $S + h$ with $R(\delta) = \frac{1}{6}V'''(\xi)h^3$. Hence $|R| \le \frac{1}{6}\big(\sup_{[S,S+h]} |V'''|\big)|h|^3$ and $\min_{[S,S+h]} V''' \le 6R/h^3 \le \max_{[S,S+h]} V'''$. Moreover $V''' = -(\Gamma/S)(1 + d_1/v)$ and $V'''' = (\Gamma/S^2)\big[(1 + d_1/v)^2 + (1 + d_1/v) - v^{-2}\big]$ with $v = \sigma\sqrt{T}$, both identical for calls and puts.*

Proof. Taylor's theorem on $u \mapsto V(u)$ between S and $S + h$ gives the first display after substituting $V' = \Delta$ and $V'' = \Gamma$; dividing by $h^3/6$ and using that V''' attains its extremes on the compact segment gives the second. For the closed forms, $\Gamma = e^{-qT}\varphi(d_1)/(Sv)$ with $\partial d_1/\partial S = 1/(Sv)$ and $\varphi'(x) = -x\varphi(x)$ yields

$$\frac{d\Gamma}{dS} = \frac{e^{-qT}}{v} \cdot \frac{\varphi'(d_1)S/(Sv) - \varphi(d_1)}{S^2} = -\frac{\Gamma}{S}\Big(1 + \frac{d_1}{v}\Big),$$

and one further differentiation, with $A = 1 + d_1/v$, $A' = 1/(Sv^2)$, gives V''''. Call/put equality for every order ≥ 2 is put–call parity, $V_{\text{call}} - V_{\text{put}} = Se^{-qT} - Ke^{-rT}$, which is affine in S.

Four consequences, each checked before being believed (`docs/analysis/delta_gamma_error_bound.md`). The *asymptotics*: fitted per direction over shrinking windows, the log-log slope of $|R|$ converges on the theoretical three from both sides, $2.9710 \to 2.9988$ down and $3.0253 \to 3.0012$ up, and at the smallest swept shock R equals $\frac{1}{6}V'''(S)h^3$ to 0.38 %. The *inequality*: 618 shocks in both directions give no violation and no vacuity, $|R|$/bound running 0.9962 to 0.2842. The *sign change*: this ladder's V''' crosses zero at spot 94.55, a 5.45 % down move, which puts the remainder's own zero at a **21.1447 %** move — inside the single 20–30 % bracket where the older artifact changes sign, data frozen in the manifest before this derivation existed. And the *fourth derivative* is confirmed a second way: the departure from cubic behaviour per unit of δ tends to $SV''''/(4V''') = 3.752361$, measured 3.759289 and 3.745379 from the two directions, which reaches V'''' through the price surface rather than through a difference table.

Three limits are recorded rather than tuned away. The arithmetic floor is a property of the book's magnitude — near $\delta = 3{\cdot}10^{-5}$ here — and an earlier draft of the analysis took it from a similar-looking single-option test, at the wrong unit and the wrong scale. A single fitted slope is not evidence of order three: pooling the two directions yields 3.037 ± 0.003, eleven standard errors from theory, where those sigma are the up/down separation appearing as curvature. And the bound bounds the *map*: it says nothing about whether the scenario is plausible. It also holds only while one factor moves, which is not how the published scenario set is written; the next subsection takes the two-factor case up rather than leaving it as a refusal (`docs/limitations.md`, items 65–66).

8.4 The same bound when two factors move

The refusal above was earned rather than assumed: `run\_scenario` maps the equity factor to second order and the volatility factor *linearly* — `out.volatility=term(vega)*absolute` — so once a scenario moves both, the terms it drops are no longer third order. Write $h = S\delta$ for the spot move and k for the absolute move in volatility, and let $g(t) = V(S+th,\, \sigma+tk)$. Taylor's theorem in the single variable t, with the chain rule applied three times, gives for some $\xi \in (0,1)$

$$V(S{+}h, \sigma{+}k) - V(S,\sigma) = \underbrace{V_S h + \tfrac{1}{2}V_{SS}h^2 + V_\sigma k}_{\text{the map}} + \underbrace{V_{S\sigma}hk + \tfrac{1}{2}V_{\sigma\sigma}k^2}_{\text{order 2}} + \underbrace{\tfrac{1}{6}g'''(\xi)}_{\text{order 3}}, \qquad (11)$$

where g''' is the third *directional* derivative along the ray, $g''' = V_{SSS}h^3 + 3V_{SS\sigma}h^2k + 3V_{S\sigma\sigma}hk^2 + V_{\sigma\sigma\sigma}k^3$, evaluated at the moving market point. The two second-order coefficients are vanna and volga, and the four needed for the bound are implemented as closed forms beside the existing Greeks, added as new value types because `Greeks` is a frozen surface. Unlike the single-factor case of §8.3, this is not primarily an inequality: $6(R - \text{order-2})$ *equals* $g'''(\xi)$, so it must lie inside the segment's own range of g''', which refutes a wrong partial as soon as one is transcribed badly.

Three things in Table 14 are worth separating, because they are three different kinds of claim. *The order* is asymptotic and holds: over four successively narrower fit windows the log-log slope of $|R|$ against shock size converges on two along every joint ray ($1.9098 \to 1.9975$ on the crash ray), while a pure-spot ray on the same book holds at 3.0009 — which is §8.3's answer, reconciled rather than displaced, and the ratio of R to the closed-form order-2 term reaches 0.9937 at the smallest scale. Convergence is asserted as an *ordering* of windows for the four

term on `risk\_off`	value	comment
re-priced P&L	−71 559.18	full Black–Scholes re-pricing of the published book
map P&L	−66 238.39	`run\_scenario`, equity −15% and +6 vol points
error R	**−5 320.79**	
$vanna \cdot hk$	−173.10	the mixed second order
$\frac{1}{2} volga \cdot k^2$	+961.06	vol convexity
whole order-2 term	+787.96	*wrong sign* at this size
$\frac{1}{2} V_{SS\sigma} h^2 k$	**−8402.16**	gamma applied at a volatility the move changed
$\frac{1}{6} V_{SSS} h^3$	+3893.86	the single-factor cubic of §8.3
interval for R	$[-6063.95, -4135.87]$	from (11); **excludes zero**

Table 14: The published `risk\_off` scenario, bounded. Its shocks are imported from `experiments/stress_testing/run.py` rather than re-typed here, so the bound is about the scenario the repository actually ships.

joint rays only, and not for the pure-spot ray: its tightest error sample is 1281× the double spacing of the $1.09 \cdot 10^5$ book value it is subtracted from, where the joint rays reach $1.3 \cdot 10^6$ to $1.9 \cdot 10^6$, so ordering those windows would compare cancellation noise — a distinction the Linux runner forced, two guards deep, recorded in `docs/limitations.md` item 72. *The inequality* holds unconditionally and is what survives at useful sizes: 132/132 swept joint shocks, out to a 30% move with +20 volatility points, show no violation, and on `risk\_off` the interval excludes zero, so the *sign* of the published error is proved rather than estimated. *The leading term* is the one that disappoints: at the size the scenario actually uses, the order-2 term is +787.96 against an error of −5320.79. A cubic term dominates it by 10.7× — or 7.4× measured against the quadratic's two contributions summed in absolute value, which is the reading `experiments/two_factor_error_bound/run.py` publishes beside it — and it is the $\frac{1}{2} V_{SS\sigma} h^2 k$ piece — base gamma applied across a move that has already changed the volatility on which gamma depends. Counted over the whole grid the largest omitted piece is that mixed cubic in 64 cells, the pure spot cubic in 36 and volga in 32, while $vanna \cdot hk$, the term the order argument singles out, is largest nowhere.

That ranking is a result about engineering priority and it is book-specific, which is the limit worth stating. On this ladder the two second-order contributions nearly cancel because the K=90 and K=110 legs carry vanna of −5089.09 and +5628.57, leaving 192.33; a concentrated book would not cancel and would rank differently. The transferable statements are the inequality and the method: the cheap improvement to this map is re-striking gamma at the shocked volatility, not adding vanna and volga, and the sign of an error can be established on a scenario the report already publishes. The direction in which the order-2 term vanishes is predicted in closed form at $k/h = -2\, vanna/volga$; its empirical counterpart needs a bracket whose useful width shrinks with the shock, and the probe shipped in the artifact records a coarse scan finding the root at 0.87× the prediction at one scale, 1.81× at another and not at all at a third—so neither agreement nor disagreement is claimed, and the scan is published so that refusal can be checked (`docs/analysis/two_factor_error_bound.md`; `docs/limitations.md`, items 67–71).

8.5 Re-striking gamma: the recommendation, measured

The paragraph above ends in a priority rather than a result, and a priority is the kind of claim a later revision quietly gets wrong. `experiments/restrike_gamma_map/run.py` rebuilds the map four ways — every exposure at the base market (what ships), gamma alone at $(S, \sigma{+}k)$, gamma at $(S{+}h, \sigma{+}k)$, and all three at the shocked market — and prices each through the shipped `run\_scenario` against a revaluation of the same book, so the comparison is between four maps and not between a map and a re-typed formula.

The mechanism survives contact. The difference between the re-struck and the shipped map is exactly $(\gamma(\sigma{+}k) - \gamma(\sigma))\,\delta^2$ — the *relative* move squared, because the exposure already carries S^2 — and the divided difference across k lies inside the segment's own range of $\mathrm{d}\gamma/\mathrm{d}\sigma$ built from `black\_scholes\_mixed\_third\_derivatives`, on 7 probes with a worst relative mismatch of 3.7e-15. What the recipe removes at `risk\_off` is 7258.51 of the 8402.16 named above: **86.4 %** of it, the shortfall being a finite difference across six volatility points rather than the local derivative. The error there moves from −5320.79 to +1937.72, a factor of 0.364, with the sign flipping alongside it.

The order does not change: between the two narrowest scales of each of three rays the error of all four maps falls with a local slope of 1.98-2.01, because re-striking deletes a cubic and leaves the quadratic $vanna \cdot hk + \frac{1}{2} volga \cdot k^2$ terms where they were. Window regressions over four points give 1.89-2.05 and over the whole ray 1.92-2.34; both are published in the artifact, and the pairwise number is the one quoted because it needs no window.

The limit is the finding. On 45 of 120 cells with a volatility move the re-strike does not reduce the absolute error, and on 20 of those it at least doubles it, worst factor 142. Every one of those 20 is a cell whose shipped-map error was *smaller than the term being removed*: along a δ column the error contains a k-independent piece $\frac{1}{6} V_{SSS} h^3$ and pieces proportional to k, and where they cancel the map is exact by arithmetic rather than by merit. Deleting one of the cancelling terms spends the cancellation. Read as a prediction rather than an excuse, the zeros of the third-order truncation — assembled from closed forms, nothing fitted to the measured errors — agree with the zeros of the priced error to 0.0002-0.0024 in vol move within $|\delta| \le 0.05$, against a gated tolerance of 0.005, and drift to 0.093 by $|\delta| = 0.30$; at $\delta = +0.30$ the truncation predicts a zero the priced map does not have, and across the twelve columns it finds 14 against the priced error's 13. A third-order truncation is an argument near the base market, which is where the published scenarios live, and stops being one out at a 30 % move.

Re-pricing *every* sensitivity at the shocked market — the fix a reader reaches for first — is worse than the gamma-only variant on 116 of the 120 cells, and on `risk\_off` its error is 60275.64 against the shipped map's 5320.79, a factor of 11: moving delta and vega replaces a linear leg with a tangent at the wrong point of the move. What survives is the specific half of the earlier recommendation, with its scope stated: re-striking gamma helps on the published scenario and on 62.5 % of the grid, reduces the value-weighted error by 0.3704, and is not a uniformly safer map. Nothing here changes `run\_scenario`; `docs/limitations.md`, item 78 records what a user who needs the residual controlled at joint published sizes must do instead.

8.6 The fourth order: a wider crossing radius, and no cheaper amount

The prediction above is gated only inside $|\delta| \leq 0.05$, which is an argument about the region a third-order truncation owns rather than about the mechanism. `cpp/include/quantrisk/pricing/black\_scholes.hpp` now carries the four mixed fourth partials, each in the shape $V = P \cdot S^{1-n_{\text{spot}}} \cdot T^{n_\sigma/2} \cdot R(d_1, v)/v^3$, and `experiments/fourth\_order\_crossing\_map/run.py` adds them to the same column truncation, on the same book, with the cubic imported from the phase-15 experiment rather than re-typed.

Two checks say the added piece is what it claims. Along four joint-shock rays scaled from published size to a thousandth of it, the residual the cubic leaves falls with log-log slope 3.96–4.37 and the residual the quartic leaves with 4.73–5.01 — a fourth-order remainder against a fifth-order one — and the ratio of the two falls with slope 0.62–1.04 against `logscale`, the 1.0 an extra order implies, and is down to 0.0013–0.048 by `scale` $= 3 \times 10^{-3}$. The window is part of the result, not a convenience: the residuals are differences of book values near 1.09×10^5, so below `scale` $= 10^{-3}$ the subtraction is at the arithmetic floor and a fit that reaches down there returns 2.46–4.30 instead. Both fits are in the artifact.

What the extra order buys is place rather than amount. Within $|\delta| \leq 0.15$ the nearest zero of the quartic truncation lies inside the published tolerance of 0.005 on every column that has a priced zero, worst distance 0.00162, where the cubic's worst over the same columns is 0.01887: a factor of 11.6, and a factor of 211.6 inside the 0.05 limit v1.5.0 already gated (1.15e-5 against 0.00243). The gate is two-sided — the quartic must pass where the cubic fails — so the widening cannot be an artifact of a wider test. Beyond 0.15 both fail and the quartic fails less: 0.0323 against 0.0934 at $\delta = -0.30$. On the amount, the same extra order moves the `risk\_off` estimate of the shipped map's error from -4135.87 against a priced -5320.79 (22.3 % short) to -6182.27 (16.2 % over): closer, on the other side, and the order-four piece it adds, -2046.40, is larger than the 1184.92 the price needed. For the re-struck map of v1.5.0 the pair reads 61.2 % against 44.5 %. At full shock size on a shallow ray the quartic residual is 1.38x the cubic's with the opposite sign.

The crossing *count* barely moves: 13 zeros in the priced error, 14 found by the cubic and 14 by the quartic. The quartic repairs the cubic's three column-level mistakes — a crossing missed at $\delta = -0.05$, one invented at $\delta = +0.05$, one invented at $\delta = +0.30$ — and invents a different one, at $k = 0.164$ on the $\delta = +0.10$ column. The counts are reported rather than gated, because a tolerance on distance alone would have hidden a false zero completely. Scope: one three-strike ladder, one maturity, one base volatility — a scope the next paragraph tests rather than keeps; `docs/limitations.md`, item 79 and `docs/analysis/fourth\_order\_crossing\_map.md` §5 carry the rest.

Does the radius belong to the model or to the book? `experiments/second\_book\_crossing\_map/run.py` re-measures it on four further books — a long-dated wide ladder, a short-dated tight one, a deep out-of-the-money ladder and an in-the-money mirror — after proving the re-implementation is the shipped machinery wherever both are defined: ten book coefficients, both truncations and the engine P&L over 30 joint moves agree bit-for-bit on the published ladder, and that ladder's two radii come back at 0.05 and 0.15. The direction transfers. The quartic radius is at least the cubic's on all five books, and its worst nearest-zero distance inside the cubic's own radius is the smaller on all five, 1.15e-5 to 2.13e-4 against cubic distances of 5.36e-4 to 2.73e-3. The magnitude transfers to none of them:

the widening factors are 1.3, 2.0, 2.5, 3.0 and 3.0, and the cubic radii are 0.02, 0.05, 0.05, 0.10 and 0.15, so $|\delta| \le 0.15$ is this ladder's number and `docs/release\_notes\_v1.6.0.md` now carries a dated correction saying so. The set was built to span the ratio of the order-four spot piece to the order-three one, 0.015 to 3.887, on the expectation that a small fourth-order piece leaves an added order little to remove; the book chosen for the smallest ratio tied for the widest gain, so no mechanism replaces the one that failed. Item 81 of `docs/limitations.md` records the edges, including the two ways the radius definition was wrong before the measurement was right.

The next order exists, and its answer is partial. The residual the quartic leaves is a fifth-order one, so `cpp/include/quantrisk/pricing/black\_scholes.hpp` now carries the six mixed partials of total order five in the same shape with one further power of v, $V = P \cdot S^{1-n_{\text{spot}}} \cdot T^{n_\sigma/2} \cdot R(d_1, v)/v^4$. The numerators are not typed: `scripts/derive\_fifth\_order\_partials.py` parses them out of the source, forms the quotient of the symbolic derivative of the price by the prefactor the header claims, and meets it at 60 markets spread over the (d_1, v) plane to 8.7×10^{-58} relative worst at 60 working digits. Nothing there is fitted, so agreement at the precision floor means the two sides are one function rather than a curve through points. The degree in d_1 is $8 - n_{\text{spot}}$, which is why a basis truncated at total degree five cannot represent this family: the first attempt at that check reported residual noise where it should have reported a bad model.

That paragraph stood alone for one phase, because the capability and the claim that consumes it were delivered separately. `experiments/fifth\_order\_crossing\_map/run.py`, the eighteenth suite member, adds (1/120) of the fifth directional derivative to the column truncation and re-runs the same paired-magnitude contiguous rule on the same five books. Order five widens the radius on 2 of the 5: the published ladder from 0.15 to 0.20 and the deep out-of-the-money ladder from 0.20 to 0.30. It leaves 3 unchanged, and of those three, 2 sit at the edge of the swept grid, so their radius is unresolved rather than refuted. Inside the quartic's own radius the quintic is the closest of the three truncations on all 5. The mechanism that would order the outcome is absent again: the ratio of the order-five spot piece to the order-four one at the measure column spans 0.057 to 0.588 across the set, and the book with the *smallest* of those ratios took the larger widening.

A sixth book, and what a rule can reach. The ratio axis is extended by rule rather than by choice: `experiments/sixth\_book\_crossing\_map/run.py` fixes a 23-candidate grid, ranks it by the order-five-to-four piece ratio at the same measure column, and takes the first candidate whose radius is *defined* — at least three swept columns carrying a priced crossing — reading no radius until the choice is made. The winner sits 24-fold below the previous minimum, at ratio 0.0024, and its quartic radius is already 0.3, the widest swept move, so the quintic cannot rise on that grid whatever the truncation does: $0.2 \to 0.3 \to 0.3$, a factor of 1.000. The arithmetic still improves — worst column inside the quartic's own radius 0.000389 against the quartic's 0.000461 — and across the six books the quintic widens 2, leaves 4 unchanged and thins none, with 3 now at the grid edge. The five earlier radii come back digit for digit from Phase 20's artifact, and the scan is published with the result (`experiments/sixth\_book\_crossing\_map/results/sixth\_book\_crossing\_map.json`), so the rule can be audited rather than trusted. Item 88 of `docs/limitations.md` states what the sixth book therefore does not say.

Items 82 and 83 of `docs/limitations.md` keep the arithmetic, the measurement and the two refusals apart, and `docs/analysis/fifth\_order\_crossing\_map.md` prints the per-book radii the claim is read from.

9 Numerical Validation

Validation is organised around three levels defined in `docs/validation_protocol.md` before any code existed, and executed per phase with real output.

- **Level 1 — analytical identity.** Parity identities, degenerate limits, $ud = 1$, the no-early-exercise theorem, control-variate exactness on an affine payoff, the geometric-average variance limit, constraint residuals that must vanish, and analytic Greeks against central differences with a Richardson error estimate instead of a tuned tolerance.
- **Level 2 — live independent oracle.** QuantLib's analytic and lattice engines [the QuantLib contributors, 2024], SciPy [Virtanen et al., 2020], and for the portfolio layer PyPortfolioOpt (behind SciPy's SLSQP) [Martin, 2024], cvxpy with OSQP and SCS [the cvxpy development team, 2024], and scikit-learn [Pedregosa et al., 2011]. Oracle versions and configuration are printed into each artifact; no oracle output is pasted into a test.
- **Level 3 — statistical.** Convergence order fitted on log-log axes with standard errors; interval coverage against the exact binomial band; size and power of coverage tests; rolling-origin portfolio cost measured against a forward covariance known in closed form.

9.1 How tolerances are set

Both absolute and relative error are reported for every comparison, and tiers follow the source of the bound: deterministic analytic checks are tight, pinned in the Phase 2 test files and justified by double rounding rather than trial and error; lattice and Monte Carlo bounds are looser and justified by theory; statistical checks are hypothesis-test based (coverage within a binomial interval, backtest p-values, slope tolerance bands) (`docs/validation_protocol.md` §2). Two prohibitions do most of the work: lowering a tolerance or deleting a failing test to make CI green is forbidden, and expected values must be *derived*, never transcribed.

The derived-values rule was earned. Phase 5 caught two invented constants in its own tests: a chi-square 5 % point for 4 degrees of freedom filed under 5 degrees of freedom, and a tail probability that contradicted the closed form $\exp(-x/2)$ for 2 degrees of freedom. Both looked plausible; both were wrong. Critical values are now solved by `chi_square_isf` from the same survival function that produces the p-value, so a statistic and its threshold cannot drift apart by someone mistyping a table (`docs/validation_protocol.md` §2; `docs/model_cards/var_es.md`).

9.2 Validation matrix

Table 15 is the Phase 10 matrix required by `PROJECT_SPEC.md`, organised by the artifact a number came from. The project's own version of the same table, which adds the bound each *test* asserts next to the error actually measured and labels a bound chosen by observation as *(measured)*, is `docs/validation_matrix.md`; its status vocabulary distinguishes `validated`, `validated(L1only)`, `partiallyvalidated` and `weakerthanclaimed`. Status values in Table 15 are the phase-gate verdicts recorded in the reports named there, and no global test-coverage or pass-rate figure is asserted anywhere in this report beyond the counts the phase reports themselves record.

Three rows of that matrix carry caveats that the table form flattens, and they are worth restating. Rho's relative error of 1.07×10^{-7} occurs on values passing through zero, where the absolute error is 5.12×10^{-13}, which is why relative errors are reported only above the floor the artifact records. The binomial row asserts a *rate*, not a value: a 1.82 gap at 50 steps (`benchmarks/quantlib/results/pricing_vs_quantlib.json`) is the $O(dt)$ truncation the model card derives, and the falsifiable claim is the fitted slope. And the stress engine's oracle bounds the arithmetic, not the scenario: re-pricing the same book through Black–Scholes shows that the map is a correct second-order expansion and how fast that stops being true, but it says nothing about whether a chosen scenario is the right one to run, because there is no independent implementation of a management assumption (`docs/validation_matrix.md`).

9.3 Defects found by the process

Reporting these is not an apology; it is the part of the evidence with the most information in it. A validation regime that reported nothing across Phases 1 to 9 would more plausibly have been measuring the wrong thing.

Lanczos denominator off by one. In `cpp/src/math/special.cpp`, the series denominator used $z+i-1$ where the definition requires $z+i$. The symptom was $\log\Gamma(13) = 20.078$ against 19.987, and `log_gamma(0.25)` returning NaN — the log of a negative sum. After correction the implementation agrees with `math.lgamma` to 1.8×10^{-15} maximum absolute error (`docs/phase_reports/phase-05-market-risk.md` §5).

A likelihood term that made a likelihood ratio negative. The Kupiec statistic dropped $x \ln p$ and used $x \ln x$ in *both* likelihoods. The result reduced to its first term and gave -0.9989 for 13 violations in 250 days — a likelihood ratio cannot be negative — with the p-value pinned at 1.0, so a badly under-set VaR was reported as a pass. Both binomial likelihoods were rewritten from the definition, and a $0 \log 0$ limit helper replaces the term that hid the bug (`docs/phase_reports/phase-05-market-risk.md` §5). This is arguably the most important defect in the project: it made a risk-model validation test incapable of failing.

A quantile read from an unsorted array. `monte_carlo_var` computed its quantile from an unsorted loss array, reporting -0.41 for the VaR of a standard-normal P&L sample where the population value is about 1.64; the confidence interval was computed on the same

Table 15: Validation matrix. "Tolerance/bound" is either the bound asserted in the named test or the measured value where the artifact records one. Every row's evidence is a committed artifact.

Component	Level and method	Oracle	Measured result
Normal distribution, inverse, statistics	L1 identities; L2 dense grid	SciPy [Virtanen et al., 2020]	max abs. err $< 10^{-12}$; L3 KS on 50,000 normals, χ^2 on 80,000 uniforms
BS price & Greeks	L1 parity and limits; L2 live repricing	QuantLib analytic	Tables 1; parity 8.00×10^{-15}
CRR lattice	L1 $ud = 1$, no-early-exercise; L2 lattice engine; L3 slope fit	QuantLib lattice engine	Table 2; corner gap read as normalised property
MC European	L1 exactness identities; L2 analytic limit; L3 slope & coverage	QuantLib analytic + NumPy/PCG64	Tables 4; slopes -0.61 to -0.51; 12/12 coverage
Variance reduction	L3 out-of-sample MSE over seed ensembles	— (own estimator)	1.12–2.77× antithetic, 1.93–40.47× control variate
Asian (geometric)	L1 variance limit; L2 analytic	QuantLib	5.5×10^{-4} (day rounding)
Barrier	L2 continuous analytic; L3 monitoring refinement	QuantLib barrier engine	Table 3 rows 3–4
Heston	L1 $\xi = 0$ collapse; L2 semi-analytic; L3 step refinement	QuantLib semi-analytic Heston engine	2.1×10^{-3} / 4.5×10^{-3}, weaker than BS by construction
VaR / ES estimators	L1 definition; L2 SciPy quantiles and quadrature; L3 realised rates	SciPy	Table 5
Bootstrap intervals	L3 coverage vs exact binomial band	own exact intervals	Table 7
Kupiec / Christoffersen	L3 size, power, discrimination	chi-square from own special functions	Table 6
Covariance estimators	L2	`numpy.cov`, scikit-learn	7.2×10^{-16} rel; 7×10^{-19} abs
Portfolio solvers	L1 certificates; L2 two solver families	PyPortfolioOpt, cvxpy	Table 9
Stress mapping & attribution	L1 residuals; L2 NumPy/SciPy and full BS revaluation; L3 MC stress VaR	Black–Scholes engine, NumPy, SciPy	Tables 11, 12; residuals 0.0
Python/C++ consistency	exact match against a compiled reference tool	own binary	`rel=0,abs=0` on 256 normals and more
Data layer	provenance integrity and refusal behaviour	live responses	13/13 fixture digests match; socket-blocking test passes

garbage. The fix sorts first, and the `*_sorted` primitives now refuse unsorted or NaN input rather than returning a plausible number (`docs/phase_reports/phase-05-market-risk.md` §5).

Catastrophic cancellation in an incomplete gamma. The lower regularised gamma was computed as $1 - (1 - P)$, so $P(20, 0.01) = 4.07 \times 10^{-59}$ came out as exactly 0.0. Both tails are now evaluated from the representation that is not sitting at one (`docs/phase_reports/phase-05-market-risk.md` §5).

Comparing against a rounded oracle output. The first portfolio benchmark compared weights against PyPortfolioOpt's `clean_weights()`, which rounds to five decimals and zeroes anything under 10^{-4}. That floored every disagreement at about 5×10^{-6}, so the test was measuring the oracle's rounding rather than the optimisation — and the tests carried the same defect behind a comment claiming 10^{-9} agreement while asserting 5×10^{-5}. Reading raw solver output tightened the quadratic comparisons by roughly seven orders of magnitude (`docs/phase_reports/phase-06-portfolio-optimisation.md` §6). A related trap in the same area: scikit-learn's `EmpiricalCovariance` is the biased maximum-likelihood estimator with divisor T while this project's is unbiased with $T - 1$, so comparing them returns about 6×10^{-6} on a 400-observation window — a convention difference that looks exactly like a bug. The reference has to be `numpy.cov(ddof=1)`, and the test says so in its own name.

A continuity correction implemented backwards. As described in Section 4, the barrier correction initially moved the effective barrier away from the spot, and the test asserted the wrong inequality and passed, because the test shared the misconception. The oracle comparison is what caught it; the tests now assert the tightened barrier, the cheaper price and the reduced error (`docs/model_cards/path_dependent_options.md`; `docs/phase_reports/phase-04-path-dependent-heston.md`).

An empty sensitivity vector sizing a beta. The stress layer's portfolio beta was once sized from whichever exposure block happened to be first. For a book whose only sensitivity is vega, the `delta` vector is empty, and the beta came out empty — which allocated exactly zero risk to a position that carried real risk. The function now takes the factor count, and the reason is recorded in the code comment and the model card (`cpp/src/stress/engine.cpp`; `docs/model_cards/scenario_stress_testing.md`).

A test runner that hid unexecuted tests. `ctest--presetdev` uses `stopOnFailure: true`, and Phase 5's cases are numbered after the earlier ones, so a run that halted printed "99 % tests passed, 1 failed out of 94" while 16 later cases had never executed. Running the Catch2 binary directly exposed 6 failing cases and 11 failing assertions. Every phase report from that point quotes the Catch2 case count and the CTest total together, because the discrepancy is what to look for (`docs/phase_reports/phase-05-market-risk.md` §5, §8).

Provenance that named the wrong commit. `git_commit` was captured at configure time, so every artifact from Phases 1–4 named the *previous* phase's commit while the working tree held the real change — and a clean tree turned that into a confident false statement.

Artifacts now record run-time `git_commit`, `binary_git_commit`, `working_tree_dirty` and `uncommitted_paths`, plus a provenance sentence that differs for the four reachable cases; `artifact_manifest` also stopped storing absolute paths, which had been publishing a developer's home directory in committed evidence files (`docs/validation_protocol.md` §3; `docs/phase_reports/phase-05-market-risk.md` §5).

Two defects in the evidence tooling and one in the CLI. A `!!`-prefix check applied to the path instead of the status column was a dead branch written from a wrong model of `gitstatus--porcelain`, caught by the first tests written for the parser (`docs/phase_reports/phase-05-market-risk.md` §5). The CLI's analytic-versus-finite-difference check compared against a guessed 5×10^{-5} gap and failed at 6.39×10^{-5}, which is the correct truncation error; it was replaced by an assertion about the ratio of successive gaps rather than relaxed (`docs/phase_reports/phase-09-research-api.md` §6).

Defects outside the numerics. An unreachable target return produced `feasible=false` with a correct numerical violation field, but a human-readable note describing only the binding target; the note now states the bound violation, guarded by a test that was falsified before being trusted — with the pre-fix source rebuilt, it fails at `tests/cpp/test_mean_variance.cpp:317` (`docs/phase_reports/phase-06-portfolio-optimisation.md` §5). pybind11's value semantics made `exposures.delta[0]=x` edit a copy and silently lose the write, which cost an hour of chasing an "engine returns zero" bug and is now pinned by a named test (`docs/phase_reports/phase-07-stress-testing.md` §8). Live EDGAR responses send `"fy":null`, so `row.get("fy",0)` returned `None` and `int(None)` raised on real data — a failure mode a hand-authored fixture could not have produced (`docs/phase_reports/phase-08-public-data.md` §6). The CFTC paths commonly cited in examples return 301 then 404; the working path was read off the agency's own index, and the parser keys off the file's header rather than column position (`docs/phase_reports/phase-08-public-data.md`). A bare `cmake--presetdev` with no active virtualenv silently built the extension against a different Python than the one running the tests; the documented commands now use `uvruncmake` and configure prints the interpreter it found (`docs/phase_reports/phase-05-market-risk.md` §4; `README.md`).

An inconsistency found while assembling this report. The variance-reduction section of `docs/model_cards/monte_carlo_gbm.md` quotes ranges of 1.12–1.9× (antithetic) and 1.9–2.3× (control variate) and paths-per-second figures of about 45.6M, 5.0M and 86M, while the raw artifacts record 1.1155–2.7741 and 1.9313–40.468 and mean rates of 46,210,785, 5,728,661 and 110,122,785 (`experiments/variance_reduction/results/summary.json`; `benchmarks/performance/results/monte_carlo_speed.json`; `docs/phase_reports/phase-03-monte-carlo.md`). This report cites the raw artifacts, which are regenerated by the recorded commands, and records the model card as stale rather than reconciling it by hand.

9.4 What the process did not catch

Level-1 and Level-2 validation shows that an implementation agrees with a formula and with other implementations; it cannot show that the formula describes the world. Three gaps

Table 16: Terminal-only European Monte Carlo at 200,000 paths, 7 repetitions, single thread, on the recorded arm64 platform. The price column is the estimate each implementation returned, so the timing comparison cannot be made by implementations that price something else. Source: the run in `benchmarks/performance/results/monte_carlo_speed.json` dated 2026-10-09T04:54:37+00:00; the manifest binds that file to the revision and toolchain it was produced with. Earlier dated runs of the same script are in `docs/phase_reports/phase-03-monte-carlo.md` and `benchmarks/suite/results/suite_headline.csv`.

Implementation	mean s.	std. s.	paths/s	estimate	z vs analytic
C++ core	0.005123	0.000151	39,040,079	11.14596	+0.537
pure Python loop	0.052799	0.012910	3,787,943	11.11229	−0.292
vectorised NumPy	0.002536	0.000286	78,860,106	11.11430	−0.242

are worth naming. The reference tool did not emit prices in Phase 2, so the pricing tests lean on independent recomputation and on QuantLib instead — a stronger path, but an asymmetry the manifest should record (`docs/phase_reports/phase-02-deterministic-pricing.md` §8). The Heston oracle is itself numerical. And a passing comparison can encode a convention error shared by both implementations, which is exactly what happened with the barrier direction and with `clean_weights()`.

10 Performance Evaluation

The project records one performance measurement and describes its scope precisely enough that the measurement cannot be over-read.

The measured ratio of C++ to a pure Python loop is 10.31× in the run tabulated here, whose absolute throughputs run from the 21,130,162 paths/s a loaded desktop has produced to the 46,382,828 the quietest committed run gives, and this one sits between them at 39,040,079, which is why the claim is the ratio and not the rate; against vectorised NumPy it is 0.495×, that is, NumPy is faster in this regime, and the artifact says so (`benchmarks/performance/results/monte_carlo_speed.json`; `docs/phase_reports/phase-03-monte-carlo.md`). The claim is scoped by four recorded caveats in the same artifact. The work measured is a terminal-only draw, one normal per path, which is exactly the case a vectorised generator is built for; the C++ core's advantage lies in per-path state — running maxima and running averages for barrier and Asian payoffs — and in holding $O(\text{paths})$ memory instead of $O(\text{paths} \times \text{steps})$, neither of which this benchmark exercises. Both sides are single-threaded: no OpenMP or TBB in the core, no multiprocessing in the baseline, so no multicore claim is available from these numbers (`docs/limitations.md`, item 16). The pure-Python baseline uses `random.Random(seed).gauss`, a different generator with pair caching, so the comparison is cost per path and not equality of estimates (`docs/limitations.md`, item 21). Timings are wall-clock from `perf\_counter`, one process at a time.

The ratio is less stable than its digits suggest, and the project's own freeze runs show why that matters. Twenty-two committed re-freezings of this identical script on this machine give a C++/pure-Python ratio of 7.77×–10.31× — a 32.6% spread — and a C++/NumPy

ratio of 0.42×–0.51×, a 20.8% spread on a quantity whose sign is the only point of reporting it. The NumPy variance is the more instructive number: the baseline itself ranged from 110M to 46M paths per second between runs, so any ratio against it inherits that movement rather than measuring the core. Nothing qualitative turns on which figure is quoted, and that is the only sense in which the conclusion is robust: the C++ engine beats an interpreted loop by roughly an order of magnitude and loses to a vectorised NumPy draw for terminal-only payoffs. What the runs do establish is the protocol's own rule that a performance number is a record of one dated run rather than a property of the machine — the manifest binds each headline value to the file, the commit and the timestamp it was read from, and `benchmarks/suite/results/suite_headline.csv` records the JSON key path behind every figure the suite summary prints.

The benchmark protocol's requirement for a performance claim — compiler, CPU architecture, path count, repetitions, mean and standard deviation, and a baseline measured on the same machine — is met by the artifact's fields, and the standard errors of the three price estimates in Table 16 are the reported ones, so the run also demonstrates that all three implementations agree with the analytic reference 11.12376 within about half a standard error (`benchmarks/performance/results/monte_carlo_speed.json`; `docs/validation_protocol.md` §4).

Memory, not throughput, is where the path-dependent layer is actually constrained: `price_path_payoff` materialises the full path matrix, which at 200,000 paths and 1,000 steps is about 1.6 GB, and no streaming payoff accumulator exists yet (`docs/phase_reports/phase-04-path-dependent-heston.md` §8; `docs/limitations.md`, item 18). `Rng::standard_normal_vector` still allocates per call, finite-difference Greeks price the option five times per Greek, and the maximum-Sharpe search costs about 100 quadratic-program evaluations per call (`docs/phase_reports/phase-02-deterministic-pricing.md`; `docs/phase_reports/phase-06-portfolio-optimisation.md`). The last of these was deliberately not rewritten: the current answer is certified and agrees to 1.2×10^{-11} in objective, so the reformulation would buy speed and coordinate tightness, not correctness.

Determinism is reported where it was measured: the portfolio benchmark's agreement block and all three of its experiment CSVs are byte-identical across independent runs, and all four stress artifacts are byte-identical between executions (`docs/phase_reports/phase-06-portfolio-optimisation.md` §5; `docs/phase_reports/phase-07-stress-testing.md` §5).

11 Limitations

`docs/limitations.md` holds 96 numbered entries, grouped by the phase that introduced them, with the rule that entries are never removed because a later phase shipped. This section summarises them faithfully; the file remains the authority.

No number in this report describes realised markets. Every quantitative result above is computed from a synthetic data-generating process written by the same author as the code: a Gaussian or Student-t return process for the risk layer, a two-regime factor process for the portfolio layer, a 10-factor fixture for the stress layer, and parameter grids for the pricing layer. The artifacts say so in their own fields — the stress study records `market.kind=synthetic` with the statement "No market data is used

or implied", and the portfolio study's caveats close with "every number describes synthetic returns from a process chosen by the author; no claim is made about realised markets" (`experiments/stress_testing/results/stress_testing_study.json`; `experiments/portfolio_optimization/results/portfolio_optimisation_study.json`). The risk layer is recorded as validated on synthetic data whose truth is known, so that nothing in it licenses a claim about realised markets and non-stationarity is out of reach by construction (`docs/limitations.md`, item 28). The public data layer, which is the one component that touches real responses, is consumed by no number anywhere in the project (`docs/limitations.md`, item 49).

11.1 Distribution of the register

Phase 1 (items 1–9), Phase 2 (10–15), Phase 3 (16–21), Phase 4 (22–27), the risk layer (28–35), Phase 6 (36–42), Phase 7 (43–48), Phase 8 (49–52) and Phase 9 (53–55). One record-keeping defect is worth naming rather than repeating: the eight risk-layer items 28–35 are attributed to Phase 5 by `docs/phase_reports/phase-05-market-risk.md` but sit under the Phase 4 heading in `docs/limitations.md`, which carries no Phase 5 section, so a reader counting entries per phase from that file alone would misattribute them.

Two further absences are structural rather than items in the register. The validation protocol defines a component-by-level matrix that expects Heston to be stated as weaker if no closed form is available, and the implementation honoured that rather than manufacturing confidence; similarly, the specification's out-of-scope list — American closed forms, discrete dividends, term-structure rates, stochastic rates, jumps, transaction-cost-aware hedging, intraday microstructure — is deferred, and adding any of it requires extending the specification first (`docs/mathematical_specification.md` §11; `docs/project_scope.md`).

Where the release engineering is incomplete, this report says so rather than anticipating it. The manifest is committed and verified: 95 artifacts, 7,211,281 bytes, of which 8 correctness benchmarks, 1 performance benchmark, 66 statistical experiment files, 4 suite-aggregate files and 16 offline fixtures, generated from a clean tree with no warnings (`evidence/manifest.json`). What remains incomplete is distribution rather than evidence: the release is a tagged public repository with assets and a `CITATION.cff` naming it, and there is no package on an index and no DOI, so nobody can install it by name and a citation cannot be resolved through a registry (`docs/limitations.md`, item 57). The fifth category is itself a Phase 11 correction: the suite's own roll-up is what `README.md` quotes for its member count, and until that phase the roll-up sat outside the chain that exists to protect quoted numbers. The sentence it quoted said twelve members, which was true when `docs/phase\_reports/phase-11-real-data-risk-study.md` wrote it; the suite has grown by six members since, and nothing in the gate set read that sentence until the drift was found by comparing this report's own two statements of the manifest's size.

12 Reproducibility

Reproducibility here means something narrower than it usually does.

Table 17: The load-bearing limitations, by layer. Quoted item numbers refer to `docs/limitations.md`.

Layer	Limitations that bound the conclusions
Foundations	Reproducibility is per-platform, not cross-platform (1). Inverse normal degrades above $\|x\| \approx 37$ (2). Sorted-input helpers trust the caller (3). Build fetches Catch2 and pybind11 over the network (5). CI runs on `ubuntu-latest` only, so the development platform is not exercised by CI (6). mypy is advisory (7).
Pricing	No term structure, one flat r and q, no discrete dividends (10). Lattice is $O(dt)$ with no extrapolation (11). Coarse-tree early exercise is biased low and the $\sigma = 0$ American value comes from a 1001-point search (14). `validate()` guards the domain, not the economics: negative rates are accepted and the lattice reports an out-of-range p in `note` instead of refusing (15).
Monte Carlo	Single-threaded; no quasi-Monte Carlo, Brownian bridge or stratification (16). Intervals are CLT approximations whose coverage was measured on light-tailed payoffs, and for heavy-tailed payoffs the same reporting would need re-examination (17). Path memory (18). In-sample control variate coefficient (19). No QuantLib MC engine available in this build (20).
Path-dependent & Heston	No calendar or business-day convention for fixings (22). Discrete monitoring is biased upward, and the correction is asymptotic (23). Monitoring grid is the diffusion grid (24). Heston validation is weaker than the Black–Scholes section because the oracle is itself numerical (25). Full-truncation Euler is biased (26). No Heston Greeks or calibration (27).
Market risk	Synthetic data only (28). Sparse tail order statistics at 99 % (29). Gaussian estimators demonstrably wrong for fat tails, in both directions (30). Percentile intervals not bias-corrected (31). *Neither* bootstrap design reaches nominal coverage under GARCH clustering (32). Unconditional estimand (33). Coverage tests cannot validate the generating model, and carry no multiple-comparison correction (34). No Cornish–Fisher, EVT, filtered historical simulation or ES backtest (35).
Portfolio	Long-only and fully invested only (36). No expected-return model of any kind (37). Single-period, so nothing is a P&L claim (38). EWMA centred on zero (39). Estimator ranking is a property of a process with a step break and no volatility clustering, which is the case EWMA was built for and did not get, so it is not a verdict against RiskMetrics (40). The simplex is dense and exponential in the worst case (41). Covariance instability is detected, refused and measured but not solved, and the binding failure — non-stationarity, at 1.57–1.60× cost — is addressed by no estimator here (42).
Stress	Delta–gamma only, with error non-monotone in move size, so no single validity range is claimed (43). Gaussian scenario moves (44). Decomposition telescopes only for Gaussian measures (45). No term structure, so no butterfly or steepener; credit is one spread (46). No reverse stress testing (47). The rank-stability result is a property of one fixture's starting spread (48).
Data & interface	No analysis consumes the data layer (49). One filer per concept (50). Six requests per company, no bulk endpoint (51). The CFTC fixture is a three-date excerpt and must not be used as a positioning dataset (52). Facades cover the documented entry points, not the whole core: Heston, Asian, barrier and the generic LP are reachable only through submodule functions (53). `validate` checks seven identities, not the suite (54). `benchmark` measures one machine and deliberately compares against nothing (55).

12.1 Per-platform, not bit-identical everywhere

The consequence for the *evidence chain* is stated rather than glossed: the frozen manifest hashes artifacts produced on one platform, so `verify\_evidence\_manifest.py` is a same-platform tamper check. A fresh run of the real-data study on the Linux runner disagreed with the macOS-committed artifact in the last digits of eight float fields — spreads of about 10^{-14} relative, for example 0.0006136533643794436 against the committed 0.0006136533643794354 — while every integer, every boolean and the whole covariance ranking matched exactly. Byte equality of a result field across libms was never a claim, and CI therefore verifies *execution* on Linux rather than byte equality there (`docs/limitations.md`, item 64).

The two-factor study of §8.4 pushed the same point one level deeper, because its published numbers are not results with small noise but *statistics fitted to* cancellation residues: the error at the tightest fit sample of the pure-spot ray is 1281× the double spacing of the book value it is subtracted from, against 1.3e6–1.9e6 for the joint rays. Four successive reproduction failures measured that — 8e-7 in a fitted slope under plain equality, 3.2e-3 under 10^{-5} relative slack, 6.6e-3 under slack derived from the artifact's own conditioning, and 39% in a regression's standard error, which is a diagnostic *of* the looseness and so could never appear on a list of exempted field names. What ships instead is a declaration rather than a tolerance: the experiment writes `reproduction\_policy.conditioning\_limited` into its own artifact, and the reproduction check compares *shape* — key sets, list lengths, types, with integers and verdicts still exact — inside those families, and value everywhere else. The claims those families carry are proved by re-derivation in the test's own environment and by the experiment refusing to publish a number outside its band, not by agreement with the laptop that produced the committed bytes (`docs/limitations.md`, items 73–75).

Identical (seed, N, parameters, code version) produces bit-identical output *on the same platform family*; it does not across libms, because `log`, `sqrt` and `erfc` are not required to be bit-identical between implementations. The project states this as the first entry in its own limitations register and answers it by making the platform part of the result: every artifact carries `build_metadata()` (`docs/limitations.md`, item 1). Determinism claims in this report are therefore scoped: the Monte Carlo layer asserts bit-identical estimates for a fixed seed in both C++ and Python, and the experiment artifacts assert byte-identical regeneration, on the recorded AppleClang 21.0.0 / arm64 / macOS platform (`docs/phase_reports/phase-03-monte-carlo.md` §9; `docs/phase_reports/phase-06-portfolio-optimisation.md` §5).

The related design choice is that the normal transform is the project's own Marsaglia polar implementation rather than `std::normal_distribution`, precisely because the latter's output is implementation-defined between libstdc++ and libc++ (`docs/phase_reports/phase-01-engineering-foundation.md` §2). This buys a reproducible stream at the cost of one more piece of numerics the project must and did validate — KS and chi-square uniformity tests on 50,000 normals and 80,000 uniforms, and moment checks inside five standard errors.

12.2 What is recorded with a result

`double` throughout the core with `float` excluded from numerical paths; no `-ffast-math` or equivalent reassociation; summation order documented where it matters (`docs/validation_protocol.md` §3). Every artifact records seeds, N, the parameter grid, the run-time and binary Git commits, working-tree state, compiler and flags, OS and architecture, library versions, a UTC timestamp, input-data SHA-256 where applicable, and a SHA-256 manifest of the files it generated. Artifact manifests list repo-relative paths, never absolute ones, so that committed evidence reads identically on every machine (`docs/validation_protocol.md` §3). The provenance distinction between the commit captured at configure time and the code that actually produced the numbers was added after Phase 5 showed that the former could be a confident false statement.

12.3 The artifact chain, and how to walk it

The required chain is script with a fixed command and seed, to raw committed result, to generated figure, to the number in the README or this report (`docs/validation_protocol.md` §5). `README.md` implements this as a claims table in which each row names the artifact it comes from, and the numbers in this report were copied from the same artifacts, in most cases from the JSON keys named beside them above. The commands are recorded verbatim inside the artifacts themselves (`command` fields), for example:

```
uv sync && uv pip install -e .
uv run cmake --preset dev && uv run cmake --build --preset dev && uv run ctest --preset dev
uv run pytest
uv run python benchmarks/quantlib/pricing_validation.py
uv run python experiments/pricing_validation/run.py
uv run python benchmarks/quantlib/monte_carlo_validation.py
uv run python experiments/monte_carlo_convergence/run.py
uv run python experiments/variance_reduction/run.py
uv run python benchmarks/quantlib/path_dependent_validation.py
uv run python experiments/var_backtesting/run.py
uv run python benchmarks/pyportfolioopt/optimisation_validation.py
uv run python experiments/portfolio_optimization/run.py
uv run python experiments/stress_testing/run.py
uv run python benchmarks/performance/monte_carlo_speed.py
uv run pytest -m oracle
uv run quantrisk validate
uv run python scripts/run_benchmark_suite.py       # re-runs the members, re-reads their output
uv run python scripts/build_evidence_manifest.py   # re-hashes into evidence/manifest.json
uv run python scripts/verify_evidence_manifest.py  # proves nothing changed since
```

The oracle-dependent steps need the optional `oracles` extra (`pyproject.toml`); everything else runs offline, and the network-dependent data layer is satisfiable entirely from committed fixtures (`docs/validation_protocol.md` §6).

The gate "clean clone → install → build → import → test" was executed in a scratch directory against the committed tree rather than the working tree, so that nothing untracked could hide a failure: 31 CTest entries and 41 pytest tests passed on the cloned revision, and the printed build metadata named the cloned commit, which is what makes the chain traceable (`docs/phase_reports/phase-01-clean-clone-run.md`). `quantriskvalidate` extends the same idea to installed wheels: it runs seven identity checks against the build a user actually has, including that the compiled extension reports the commit it was configured at, and exits 0 (`docs/phase_reports/phase-09-research-api.md` §6, §7). It is deliberately not a pytest duplicate, and it deliberately does not re-derive correctness at breadth.

12.4 Known weak points in the reproducibility story

Three should not be smoothed over. First, earlier phases' artifacts were stale in exactly the two ways Phase 5 fixed — absolute paths, and a commit stamp predating their own code — and the record states that they must be regenerated during the Phase 10 evidence freeze (`docs/phase_reports/phase-05-market-risk.md` §8). The manifest now produced for that freeze is itself evidence of the remaining gap: it records `git\_commit` as `02fea3a1847f`, `binary\_git\_commit` as `758cacb6dbc1` and `working\_tree\_dirty` as true, and its provenance sentence states plainly that it "ran from 02fea3a1847f plus the uncommitted changes" with the binary configured at the earlier commit (`evidence/manifest.json`). Provenance is only fully clean when evidence is generated from a committed tree *after* rebuilding from it, which imposes the ordering "commit code, rebuild, regenerate evidence, commit evidence", and that loop is not yet closed: the regeneration also re-measured the one timing-dependent artifact, which is why Section 10 reports both values. Second, a manifest hash proves only that a file has not changed since it was written, which the manifest itself states: "It does not prove the number inside it is right — that is what the oracle comparisons and the phase reports' test output are for" (`evidence/manifest.json`, `integrity\_note`). Third, the reference-value tool used for the exact C++/Python consistency check emits RNG, distribution and statistics values but not prices, so that one cross-language guarantee is weaker for pricing than it is for numerics (`docs/phase_reports/phase-02-deterministic-pricing.md` §8).

12.5 Reproducibility as a documentation property

The least technical and most fragile component is whether the artifacts and the prose about them stay in step. Section 9 records one such divergence found here, where a model card's summary ranges no longer matched the committed artifacts it describes. The remedy is the one the project already applies to numbers: artifacts are generated, prose cites the artifact, and stale prose is corrected by regeneration rather than by hand-editing a nicer-sounding figure (`docs/validation_protocol.md` §4).

References

Philippe Artzner, Freddy Delbaen, Jean-Marc Eber, and David Heath. Coherent measures of risk. *Mathematical Finance*, 9(3), 1999.

Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3), 1973.

Tim Bollerslev. Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 1986.

Mark Broadie, Paul Glasserman, and Steven G. Kou. A continuity correction for discrete barrier options. *Mathematical Finance*, 7(4), 1997.

Peter F. Christoffersen. *Elements of Financial Risk Management*. Cambridge University Press, 2003.

John C. Cox, Stephen Ross, and Mark Rubinstein. Option pricing: A simplified approach. *Journal of Financial Economics*, 7(3), 1979.

Steven L. Heston. A closed-form specification for market volatility. *The Review of Financial Studies*, 6(1), 1993.

International Organization for Standardization. ISO/IEC 14882:2020: Programming language C++. International Standard, 2020. Numerics library clause: the random number engines, including the 64-bit Mersenne Twister.

Wenzel Jakob, Jason Rhinelander, Dean Moldovan, and the pybind11 contributors. pybind11: Seamless operability between C++ and Python. `https://github.com/pybind/pybind11`, 2024. Pinned at v2.13.6 by the build; see docs/phase_reports/phase-01-engineering-foundation.md.

J.P. Morgan/Reuters. RiskMetrics — technical document. Technical report, J.P. Morgan, 1996. The zero-mean EWMA convention named in docs/model_cards/portfolio_covariance_and_optimisation.md.

Peter H. Kupiec. Techniques for verifying the accuracy of risk measurement models. *The Journal of Derivatives*, 3(2), 1995.

Olivier Ledoit and Michael Wolf. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2), 2004.

Robert Andrew Martin. PyPortfolioOpt: Portfolio optimisation in Python. `https://github.com/robertmartin8/PyPortfolioOpt`, 2024. Version 1.6.0; a Level-2 oracle only, and it is the SciPy SLSQP route inside that library.

Makoto Matsumoto and Takuji Nishimura. Mersenne twister: A 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1), 1998.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in Python, 2011. Version 1.9.1 in the recorded environments; its LedoitWolf and EmpiricalCovariance estimators serve as Level-2 references.

R. Tyrrell Rockafellar and Stanislav Uryasev. Optimization of conditional value-at-risk. *The Journal of Risk*, 2(3), 2000.

Florin Spinu. An efficient numerical algorithm for computing risk parity weights. *SIAM Journal on Optimization*, 23(2), 2013.

the cvxpy development team. cvxpy: A Python-embedded modeling language for convex optimization. `https://www.cvxpy.org`, 2024. Version 1.9.3 with OSQP 1.1.3 and SCS, recorded in benchmarks/pyportfolioopt/results/optimisation_vs_oracles.json.

the QuantLib contributors. QuantLib: a free/open-source library for quantitative finance. `https://www.quantlib.org`, 2024. Version 1.43; used as a Level-2 correctness oracle and imported by no library code.

the scikit-build maintainers. scikit-build-core: Python build backend for extended CMake modules. `https://github.com/scikit-build/scikit-build-core`, 2024. Required at version >= 0.10 by pyproject.toml.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 2020. Version 1.18.1 in the recorded benchmark environments.

Artifact index

The following committed files are the sole evidence for every measurement in this report.

- `README.md` — claims table naming the artifact behind each headline number.
- `docs/validation_protocol.md` — the three levels, the tolerance policy, the deterministic policy, the benchmark-integrity rules, the artifact chain.
- `docs/mathematical_specification.md`, `docs/architecture.md`, `docs/project_scope.md`, `docs/limitations.md` — conventions, layer contract, frozen surfaces, the 55-entry register.
- `docs/model_cards/` — eight cards: `black_scholes_merton`, `crr_binomial`, `monte_carlo_gbm`, `path_dependent_options`, `heston`, `var_es`, `portfolio_covariance_and_optimisation`, `scenario_stress_testing`.
- `docs/phase_reports/` — ten report files covering phases 1 to 9 (Phase 1 has two), each with its gate table and technical-debt list.
- `benchmarks/quantlib/results/pricing_vs_quantlib.json`, `monte_carlo_validation.json`, `path_dependent_vs_quantlib.json`; `benchmarks/pyportfolioopt/results/optimisation_vs_oracles.json`; `benchmarks/performance/results/monte_carlo_speed.json`.
- `experiments/pricing_validation/results/summary.json`; `experiments/monte_carlo_convergence/results/summary.json`; `experiments/variance_reduction/results/summary.json`; `experiments/var_backtesting/results/` (`estimator_convergence.csv`, `coverage_test_size_power.csv`, `bootstrap_coverage.csv`, `var_backtesting_validation.json`); `experiments/portfolio_optimization/results/` (`estimator_out_of_sample.csv`, `objective_comparison.csv`, `solver_robustness.csv`, `portfolio_optimisation_study.json`); `experiments/stress_testing/results/` (`scenario_ranking.csv`, `linearisation_error.csv`, `var_decomposition.csv`, `stress_testing_study.json`).
- `tests/python/` — the oracle-marked test modules, including `test_covariance_vs_oracles.py`, `test_stress_vs_oracles.py`, `test_normal_vs_scipy.py`, `test_cpp_python_consistency.py`.